

АННОТАЦИЯ

Исполнитель: студент группы ИМБО-02-22 Ким К.С.

Руководитель ВКР: к.ф.-м.н., доцент кафедры прикладной математики
Даева С.Г.

Выпускная квалификационная работа на тему: «Разработка модели оценки стоимости футбольных трансферов».

Работа включает в себя 11 рисунков, 6 таблиц, 2 приложения.

Список источников информации включает 37 позиций.

Ключевые слова: анализ данных, машинное обучение, футбольные трансферы, гребневая регрессия, лассо, случайный лес, градиентный бустинг, предобработка данных, прогнозирование трансферной стоимости, важность признаков.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ.....	6
1.1 Характеристика предметной области	6
1.1.1 Анализ текущего состояния предметной области	6
1.1.2 Выявление основных проблем и ограничений.....	7
1.2 Обзор существующих решений.....	9
1.2.1 Сравнительный анализ существующих решений	9
1.2.2 Выявление недостатков и ограничений аналогов.....	10
1.3 Формулировка исследовательской задачи модели.....	11
1.3.1 Определение целей, задач и критериев успешности решения	11
1.3.2 Формализация требований к разрабатываемому решению	13
1.3.3 Определение ограничений и допущений	14
2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ.....	15
2.1 Анализ и характеристика входных данных.....	15
2.1.1 Описание источников и форматов данных	15
2.1.2 Методы предобработки и очистки данных.....	18
2.1.3 Оценка качества и репрезентативности данных	19
2.2 Описание этапов разработки модели.....	24
2.2.1 Проектирование архитектуры решения	24
2.2.2 Описание математического аппарата реализуемого решения.....	25
2.2.3 Разработка вычислительной схемы	26
2.3 Обоснование выбора инструментальных средств разработки.....	27
2.3.1 Сравнительный анализ и критерии выбора программных инструментов	27
2.3.2 Описание выбранной среды разработки и требований к инфраструктуре	29
3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ.....	30
3.1 Описание реализации модели.....	30

3.1.1	Описание хода реализации	30
3.1.2	Реализация интерфейсов и взаимодействия компонентов.....	33
3.2	Анализ полученных результатов.....	34
3.2.1	Описание и реализация методологии тестирования и валидации	34
3.2.2	Сравнительный анализ и оценка эффективности с существующими решениями.....	36
ЗАКЛЮЧЕНИЕ		41
СПИСОК ИСТОЧНИКОВ.....		44
ПРИЛОЖЕНИЯ.....		48

ВВЕДЕНИЕ

Футбол давно перестал быть исключительно спортивным зрелищем, а превратился в глобальную экономическую экосистему. Объем трансферных операций достиг больших масштабов: по данным FIFA, суммарные расходы на трансферы в пяти ведущих европейских лигах в 2023 году превысили 7,2 миллиарда евро. При этом принятие решений о стоимости игроков по-прежнему основывается на экспертных оценках и интуитивных суждениях, что вызывает существенные финансовые риски и снижает эффективность распределения ресурсов. Ошибка в оценке даже одного игрока может обернуться для клуба потерями в десятки миллионов евро. В связи с этим возникает потребность в инструменте, способном давать обоснованный прогноз трансферной стоимости на основе данных об игроке.

Цель настоящей работы — разработка модели оценки стоимости футбольных трансферов, на основе игровой активности и физических характеристик игроков, использующая методы машинного обучения.

Для достижения цели были сформулированы следующие задачи:

- изучить предметную область и выявить ключевые факторы, влияющие на трансферную стоимость;
- изучить существующие решения и подходы в спортивной аналитике;
- сформировать исследовательскую задачу, определить цели, критерии успешности, требования, ограничения и допущения;
- выполнить сбор, очистку и предобработку данных;
- спроектировать и рассчитать новые аналитические признаки;
- обучить модели машинного обучения с оптимизацией гиперпараметров;
- провести сравнительный анализ эффективности нескольких алгоритмов машинного обучения;
- протестировать разработанную модель на реальных трансферах.

Объектом исследования выступает процесс формирования стоимости игроков на рынке трансферов.

Предметом исследования является математическая модель прогнозирования трансферной стоимости, на основе характеристик игровой активности, физических параметров и контекстных факторов.

В работе использованы следующие методы исследования: анализ и синтез информации, математическое моделирование, методы машинного обучения (гребневая регрессия, Lasso, случайный лес, градиентный бустинг), кросс-валидация на временных рядах, а также оценка важности признаков.

Практическая значимость работы заключается в создании автоматизированного инструмента, способного снизить субъективность при оценке футболистов и минимизировать финансовые риски клубов при заключении контрактов.

В Приложении А представлен графический материал (презентация) выпускной квалификационной работы.

В процессе написания выпускной квалификационной работы автор руководствовался следующими нормативными актами:

1. «О защите населения и территории от чрезвычайных ситуаций природного и техногенного характера» от 21.12.1994 № 68-ФЗ (с изменениями, в действующей редакции с 26.11.2024).
2. «Об основах охраны здоровья граждан в Российской Федерации» от 21.11.2011 № 323-ФЗ (с изменениями на 31.07.2025).
3. «О гражданской обороне» от 12.02.1998 № 28-ФЗ (с изменениями на 23.07.2025).
4. Приказ Минздравсоцразвития РФ от 03.05.2024 № 220н «Об утверждении Порядка оказания первой помощи».
5. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ (с изменениями на 09.02.2026).

1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

1.1 Характеристика предметной области

Футбольный трансферный рынок представляет собой сложную систему, в которой клубы передают права на игроков за плату, размер этой суммы должен отражать текущую и потенциальную ценность игрока для покупающего клуба. Оценка стоимости игроков в данной системе не является тривиальной задачей, так как формируется под влиянием множества факторов, включая игровые показатели, возраст, позицию, а также внешние рыночные условия [1.1, 1.2].

В рамках предметной области рассматриваются следующие основные вопросы:

- экономическая суть рынка трансферов;
- классификация факторов, влияющих на формирование трансферной стоимости футболиста;
- возрастные изменения стоимости футболистов;
- особенности стоимости игроков;
- влияние травм на рыночную стоимость футболистов;
- учёт времени и инфляции.

Требуется детально рассмотреть текущее состояние трансферного рынка и выявить ключевые закономерности его функционирования и определить факторы, оказывающие наибольшее влияние на стоимость игроков.

1.1.1 Анализ текущего состояния предметной области

До 1995 года действовали правила трансферов, согласно которым даже после завершения срока контракта игрока новый клуб был обязан выплатить

компенсацию предыдущему клубу. Ситуация изменилась после решения Европейского суда игроки получили возможность переходить в другие клубы бесплатно после окончания контракта [1.3]. Впоследствии цены на игроков выросли, поскольку клубы стремились продать футболистов до окончания контракта, либо подписать новые, на более выгодных условиях, стремясь не потерять актив бесплатно.

На стоимость футболиста влияет множество факторов. К ним относятся игровые показатели: количество забитых голов, количество голевых передач, процент оборонительных действий, точность передач, для защитников — отборы, для полузащитников — контроль мяча, для вратарей — «сухие» матчи и физические параметры (возраст, рост, вес, выносливость), определяющие потенциал игрока и продолжительность карьеры. Пик стоимости футболиста обычно приходится на возраст 26–29 лет, после чего она начинает снижаться. Условия контракта влияют на позиции клуба в переговорах. Если срок контракта подходит к концу, цена может быть снижена, чтобы избежать бесплатного ухода игрока. Существуют также субъективные факторы, такие как популярность в социальных сетях и успешные выступления на крупных турнирах. Молодые игроки, проявившие себя, могут подорожать, даже если их статистика игры не лучше, чем у более опытных футболистов [1.4].

Кроме того, на большинство сделок влияет общий тренд: средняя стоимость трансфера растёт, прерываясь лишь в периоды глобальных кризисов (например, пандемия COVID-19 вызвала временной спад 2019–2020 гг.) [1.5, 1.6]. Таким образом, рынок трансферов демонстрирует сложное сочетание игровых показателей, возрастных трендов и внешних экономических шоков, что необходимо учитывать при построении моделей [1.7].

1.1.2 Выявление основных проблем и ограничений

При разработке модели стоит учесть ряд проблем и ограничений.

1. Проблема с данными. Открытые источники неполные: статистика по молодым игрокам или игрокам из менее известных лиг часто отсутствует. Источники предоставляют общую статистику по матчам, но не имеют статистики по игровой активности (xG, xA, интенсивность прессинга). Коммерческие поставщики предлагают более полные наборы, но за плату.
2. Необходимость учёта выбросов и аномалий в футбольном трансфере. Топ-звезды формально являются выбросами с точки зрения статистики, но содержат важную информацию о рынке и их исключение привело бы к неспособности модели адекватно оценивать элитных игроков.
3. Временная нестабильность рынка. Рынок трансферов меняется. Суммы, которые были высокие в 2018 году, сегодня могут сильно упасть. Это связано не только с общей инфляцией, но и с изменением структуры рынка, появлением новых источников доходов у клубов, а также кризисных явлений (пандемия COVID-19).
4. Ограниченность размера выборки. Общее количество трансферов в европейском футболе измеряется тысячами, но многие из них — бесплатные переходы, аренды или мелкие сделки. Выборка трансферов с известными суммами ограничена, что создает проблемы для обучения моделей, требующих больших объемов данных.
5. Субъективные оценки. На цену игрока влияют не только показатели игры, но и работа агентов, популярность в социальных сетях, успешное выступление на крупном турнире, личные предпочтения руководства. Цена может отклоняться от его рыночной стоимости.
6. Сложность оценки физического состояния. Травмы игрока влияют на его будущую стоимость и карьеру. Необходимо учитывать не только количество пропущенных дней, но и характер травм.

Перечисленные проблемы и ограничения необходимо учитывать при построении модели. Поэтому возникает необходимость в очистке данных,

обработке пропусков и выбросов, а также в использовании методов, устойчивых к неоднородности данных.

1.2 Обзор существующих решений

В настоящее время стоимость футболистов оценивается разными способами: экспертными, статистическими и алгоритмическими. Для объективной оценки требуется провести сравнительный анализ, позволяющий выявить сильные и слабые стороны для каждого подхода.

1.2.1 Сравнительный анализ существующих решений

Экспертный подход является широко распространенным в футбольных клубах. Скауты, спортивные директора, тренеры и агенты оценивают футболистов, опираясь на опыт и интуицию. Их оценка включает просмотр матчей, анализ статистики, сравнение с похожими игроками и учет потребностей клуба. Преимущества экспертного подхода — это учет психологии, совместимости с тактикой и потенциала игрока. Однако субъективность создает проблемы, так как мнения о конкретном игроке могут отличаться.

Наиболее известная экспертная платформа Transfermarkt основана на коллективном мнении зарегистрированных пользователей (аналитиков, скаутов, болельщиков), которые обсуждают игроков и выносят вердикт о стоимости [1.8]. Учитывается все: перспективы, спортивные достижения, статистика, стиль игры, лидерские качества. Результаты зависят от коллективного мнения, что формально можно представить как функцию:

$$V = g(E_1, E_2, \dots, E_k), \quad (1.1)$$

где E_i — экспертные баллы по различным критериям, отражающие субъективные оценки специалистов.

Исследования показывают, что оценки Transfermarkt хорошо согласуются с реальными суммами сделок, что подтверждает их практическую значимость.

Другой подход реализован в исследовательском центре CIES Football Observatory, который разработал статистическую модель на основе множественной регрессии [1.9, 1.10]. Модель учитывает: возраст, срок контракта, уровень лиги, игровую статистику, результаты выступлений за клуб и сборную. Преимущество подхода в том, что модель выдаёт оценку в евро и делает это автоматически и регулярно.

Модель CIES описывается уравнением множественной линейной регрессии:

$$V = \beta_0 + \sum_{j=1}^p \beta_j X_j + \varepsilon, \quad (1.2)$$

где X_j — признаки игрока, β_j — параметры модели, ε — случайная ошибка.

В последние годы распространились методы машинного обучения такие как случайный лес, градиентный бустинг, способные выявлять сложные нелинейные закономерности [1.11–1.13].

Методы машинного обучения обладают потенциалом для более точной оценки, однако требуют большого объёма данных. При малом размере выборки возрастает риск переобучения, а также сложность интерпретации результатов. Необходимо правильно настроить гиперпараметры модели. Модели могут быть чувствительны к шуму и выбросам в данных.

1.2.2 Выявление недостатков и ограничений аналогов

Каждое из рассмотренных решений такие как экспертные и статистические методы имеет недостатки. Процесс формирования оценок на Transfermarkt занимает время, а обновления происходят редко, что может приводить к отставанию оценок от реальной формы игрока. Кроме того, эксперты могут

симпатизировать каким-то клубам, что искажает оценки, и не всегда следят за изменениями формы игрока.

Недостаток модели CIES заключается в том, что она часто завышает оценки для игроков из высших лиг и недооценивает футболистов из менее престижных чемпионатов. Это связано с линейностью зависимостей и ограниченным набором факторов. Модель плохо справляется со сложными взаимодействиями. Например, совместное влияние возраста и травм.

Для наглядности основные плюсы и минусы двух решений представлены в таблице 1.1.

Таблица 1.1 — Плюсы и минусы существующих решений

Критерий	Экспертный подход (Transfermarkt)	Статистическая модель (CIES)
Достоинства	Учет не формализуемых качеств (лидерство, медийность)	Отсутствие эмоциональной привязанности
Недостатки	Предвзятость экспертов к топ-лигам	Плохая работа со сложными взаимодействиями признаков

Вследствие этого нужно создать модель с использованием методов машинного обучения, строгой временной валидацией, богатым набором признаков.

1.3 Формулировка исследовательской задачи модели

На основании проведённого анализа переходим к постановке задачи. Данный этап позволяет задать направление дальнейшей работы и определить ключевые моменты, которым должна соответствовать разрабатываемая модель.

1.3.1 Определение целей, задач и критериев успешности решения

Цель работы состоит в том, чтобы создать модель прогнозирования стоимости футболистов, которая на основе вектора признаков игрока выдаёт

оценку его ожидаемой трансферной стоимости в евро с минимальной средней абсолютной ошибкой. Для этого сформированы следующие задачи от загрузки и очистки данных до сравнительного анализа алгоритмов и интерпретации результатов [1.14].

Задача ставится следующим образом. Пусть задан обучающий набор данных:

$$D = \{(x_i, y_i)\}_{i=1}^n, \quad (1.3)$$

где x_i — вектор признаков игрока (возраст, позиция, год и так далее),
 y_i — трансферная стоимость.

Необходимо построить функцию, которая будет предсказывать стоимость по этим признакам, минимизирующую ошибку прогноза:

$$f: X \rightarrow Y, \quad (1.4)$$

В качестве критериев успешности решения используются следующие метрики:

- MAE — средняя абсолютная ошибка в евро. Показывает, на сколько в среднем прогноз отклоняется от реальной цены.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (1.5)$$

- R^2 (коэффициент детерминации) — показывает, какую долю дисперсии целевой переменной объясняет модель. $R^2 > 0$ означает, что модель даёт информацию о разбросе цен.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (1.6)$$

- $MAPE$ — средняя относительная ошибка в процентах. Данная метрика показывает, на сколько процентов в среднем прогноз отклоняется от реальной стоимости.

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|, \quad (1.7)$$

1.3.2 Формализация требований к разрабатываемому решению

Требования к модели делятся на функциональные и нефункциональные.

Функциональные требования включают:

- приём структурированных данных об игроке, включающих возраст, позицию, рост, игровую статистику, рыночную стоимость и другие характеристики;
- предобработку признаков: масштабирование числовых переменных, заполнение пропущенных значений, логарифмирование целевой переменной,
- выдачу прогноза в евро;
- расчёт и визуализацию важности признаков для интерпретации модели.

Нефункциональные требования задают качественные характеристики модели.

- коэффициент детерминации R^2 на отложенной тестовой выборке должен быть не менее 0,5, что подтверждает способность модели объяснять значимую долю вариации стоимости;
- модель должна интерпретируемой, то есть позволять оценить вклад каждого признака в прогноз;
- воспроизводимость результатов должна гарантироваться фиксацией случайного состояния (random_state);
- устойчивость к выбросам обеспечивается логарифмической трансформацией целевой переменной и применёнными методами предобработки данных.

Сформулированные требования образуют основу для выбора алгоритмов машинного обучения и построения модели, способной решать задачу прогнозирования трансферной стоимости с заданным уровнем качества.

1.3.3 Определение ограничений и допущений

При разработке модели принимаются следующие допущения:

- используются только открытые данные;
- рассматриваются только платные переходы;
- анализируется только базовая стоимость трансфера;
- все финансовые показатели приводятся к евро;
- инфляция футбольного рынка учитывается через признак «год сезона» и строгое разделение выборки по времени (обучение данных до 2021 года, тестирующие данные на последующие сезоны).

Ограничения:

- модель не учитывает, как проходила конкретная сделка;
- не учитывает рыночные аномалии (пандемия COVID-19);
- модель работает только на тех данных, на которых обучилась;
- отсутствуют данные о точности передачи, обводках в открытом датасете.

Сформулированная задача позволяет перейти к практическому построению модели оценки стоимости футбольных трансферов.

2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

2.1 Анализ и характеристика входных данных

Необходимо подробно рассмотреть используемые источники данных, их структуру и особенности, что позволит определить возможности и ограничения дальнейшего анализа.

2.1.1 Описание источников и форматов данных

Источником данных является датасет «Football Datasets», доступный на платформе Kaggle [2.1]. Датасет включает шесть таблиц в формате csv, связанных через идентификатор игрока (player_id). Характеристики таблиц приведены в таблице 2.1.

Таблица 2.1 — Характеристики исходных таблиц датасета

Название файла	Количество строк	Количество столбцов	Описание
transfer_history.csv	1 101 440	10	История трансферов игроков
player_profiles.csv	92 671	34	Профили игроков (дата рождения, позиция и др.)
player_market_value.csv	901 429	3	Динамика рыночной стоимости
player_performances.csv	1 878 719	20	Детальная игровая статистика по сезонам
player_injuries.csv	143 195	7	Данные о травмах игроков
player_national_performances.csv	92 701	9	Статистика выступлений за сборную

Объединение таблиц производилось по идентификатору игрока с использованием операций (join) [2.2]. В результате получается одна большая таблица, содержащая 39 997 записей о трансферах с 1973 по 2026 год. На Рисунке 2.1 представлена ER-диаграмма исходных таблиц (схема связей между таблицами).

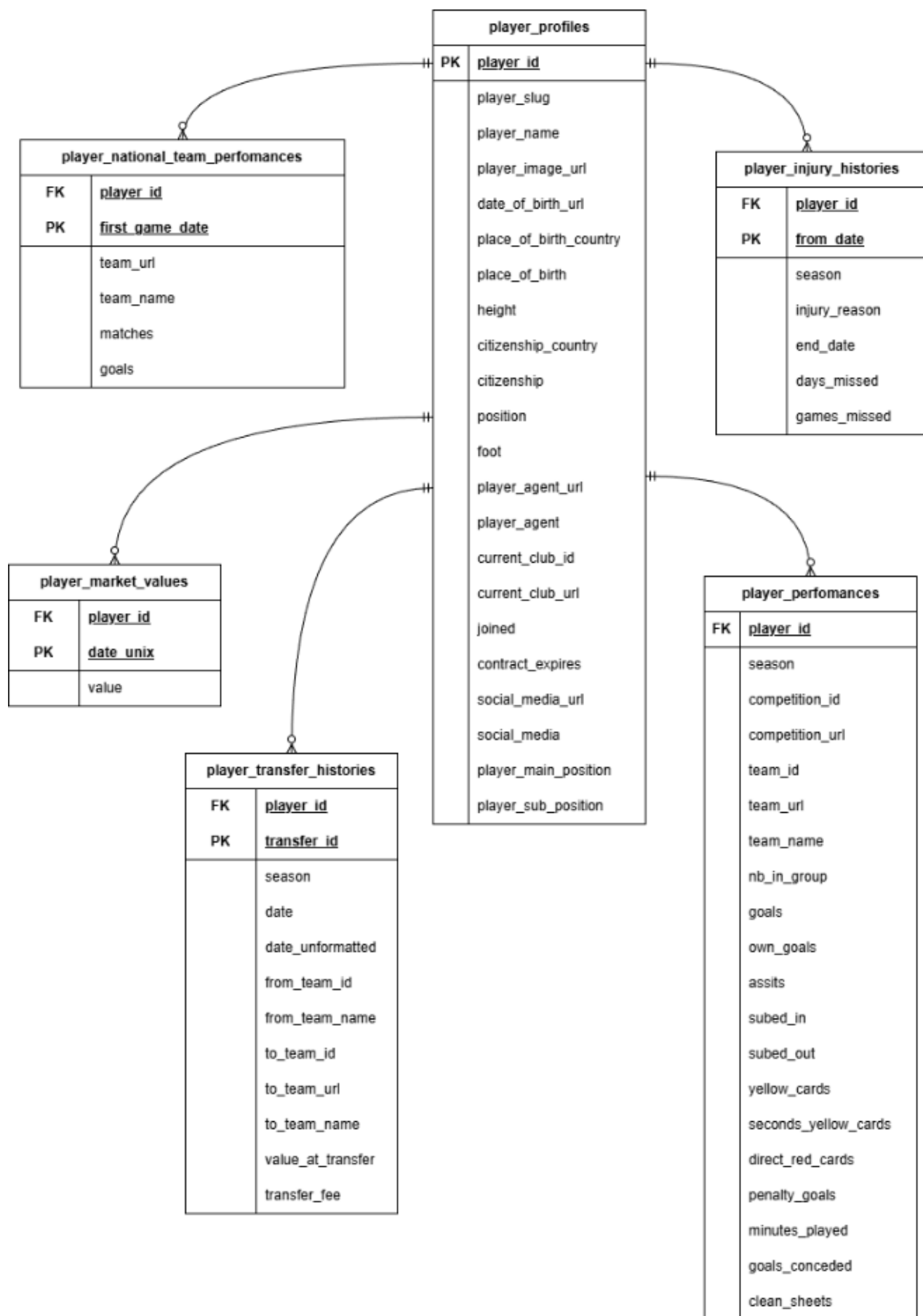


Рисунок 2.1 — ER-диаграмма исходных таблиц

Итоговая таблица содержит следующие поля:

- идентификатор игрока: (player_id — int);
- возраст на момент трансфера: (age — int);
- квадрат возраста: (age_squared — int);
- рост: (height — float);
- позиция на поле: (position_simple — object);
- тип трансфера: (transfer_type — object);
- год перехода: (season_year — int);
- стоимость сделки; (transfer_fee — int) — целевая переменная;
- логарифмированная стоимость сделки: (log_fee — float) — используется в качестве целевой переменной;
- рыночная стоимость, на момент, ближайший к дате трансфера: (market_value — float);
- суммарное количество голов за последние два сезона: (goals_last_2y — int);
- суммарное количество голевых передач за последние два сезона: (assists_last_2y — int);
- общее количество сыгранных минут за последние два сезона: (minutes_played_2y — int);
- общее количество пропущенных дней из-за травм: (injury_days — int);
- общее количество голов за сборную: (national_goals — int).

Целевая переменная — стоимость трансфера в евро (transfer_fee) — значение, которое оценивает качество прогноза модели. Распределение трансферных сумм неравномерно: большинство сделок стоят недорого, а остальные сделки стоят очень дорого. Это создает трудности при обучении модели, поэтому применяется логарифмическое преобразование (log_fee) для нормализации распределения и улучшения сходимости алгоритмов обучения [2.3].

2.1.2 Методы предобработки и очистки данных

Предобработка данных осуществлялась в несколько этапов.

Этап 1. Очистка данных, фильтрация и приведение типов.

- Преобразование денежных значений в формат float, даты — в datetime;
- Исключаются бесплатные трансферы, поскольку нулевая стоимость искажает обучение;
- Из поля season извлечён год начала сезона.

Этап 2. Логарифмическое преобразование целевой переменной.

Большинство трансферов составляют суммы до нескольких миллионов евро, тогда как дорогие трансферы являются редкими выбросами. Для улучшения свойств распределения и повышения качества обучения моделей применяется логарифмическое преобразование.

$$y' = \ln(x + 1). \quad (2.1)$$

где x — фактическая сумма трансфера в евро, y' — логарифмированная целевая переменная.

Это позволило приблизить распределение к нормальному и уменьшить влияние выбросов.

Этап 3. Обработка пропусков. Для числовых признаков применяется замена медианным значением. Для категориальных признаков заполняем модой. Для рыночной стоимости, если ближайшая оценка отсутствовала в пределах 365 дней, заполнение проводилось медианой по группе, сформированной на основе позиции и возрастной группы игрока.

Этап 4. Интеграция данных. Рыночная стоимость присоединялась с помощью `pd.merge_asof` [2.4], который подбирает ближайшую предшествующую запись, не заглядывая в будущее. Игровая статистика агрегировалась за два полных сезона до даты трансфера и добавляются дополнительные метрики:

- результативность за 90 минут считается по формуле:

$$goalsassistsper90 = \frac{goals + assists}{\left(\frac{minutes}{90}\right) + 10^{-5}}. \quad (2.2)$$

где *goals* — количество всех голов, *assists* — количество голевых передач, *minutes* — количество сыгранных минут в матче.

- голы без учета пенальти:

$$nonpenaltygoals = goals - penaltygoals. \quad (2.3)$$

где *goals* — общее количество голов, *penaltygoals* — количество голов с пенальти.

- коэффициент «сухих» матчей для вратарей и защитников:

$$clsheetratio = \frac{clsheet}{goalsconceded + 1}. \quad (2.4)$$

где *clsheet* — количество матчей, в которых команда не пропустила ни одного гола, *goalsconceded* — общее количество голов, которые команда позволила сопернику забить.

Травматичность рассчитывалась как сумма дней, пропущенных игроком до момента сделки.

2.1.3 Оценка качества и репрезентативности данных

Для оценки качества и репрезентативности данных проводился анализ распределений основных характеристик игроков и трансферных сумм [2.5, 2.6]. При этом роль подобных метрик в оценке рыночной стоимости футболистов рассматривалась в работе [2.7].

На Рисунке 2.2 представлено распределение игроков по возрастным группам. Видно, что основная масса игроков в диапазоне от 22 до 30 лет. Это пик карьеры, когда спрос игроков максимален. Медианный возраст составляет 24 года. Молодых (до 20 лет) и возрастных игроков (старше 33 лет) заметно меньше.

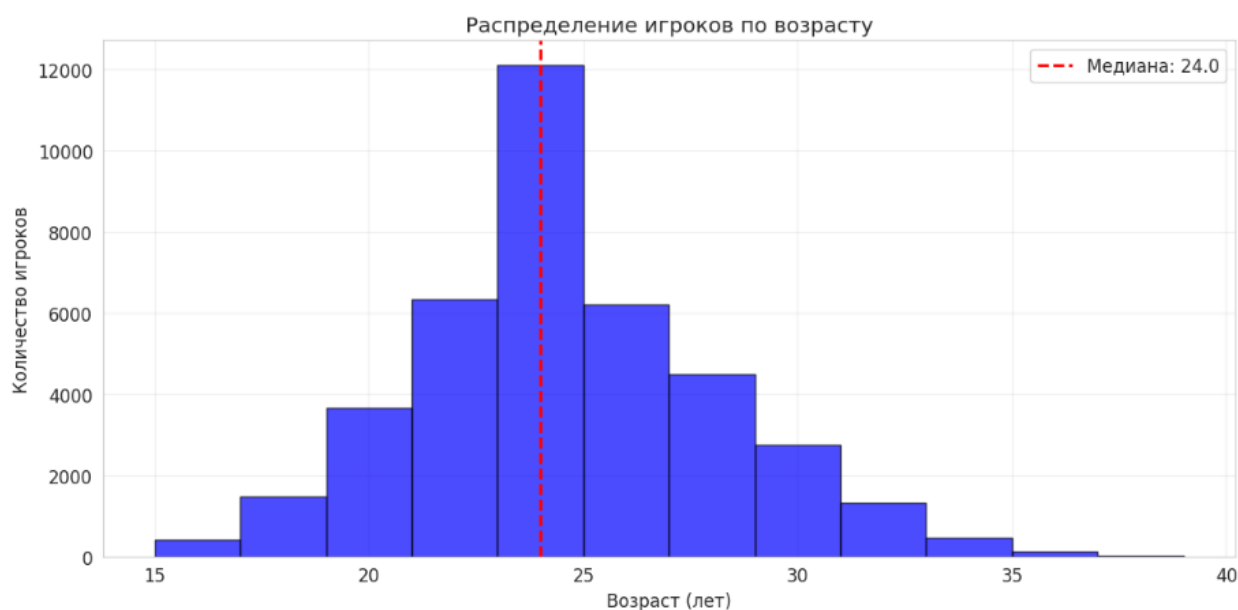


Рисунок 2.2 — Распределение игроков по возрасту

На Рисунке 2.3 представлено распределение трансферов по позициям игроков. Наибольшее число переходов совершается среди центральных нападающих, защитников и полузащитников. Вратари представлены наименьшим количеством трансферов, так как клубы редко меняют вратарей по сравнению с полевыми игроками.

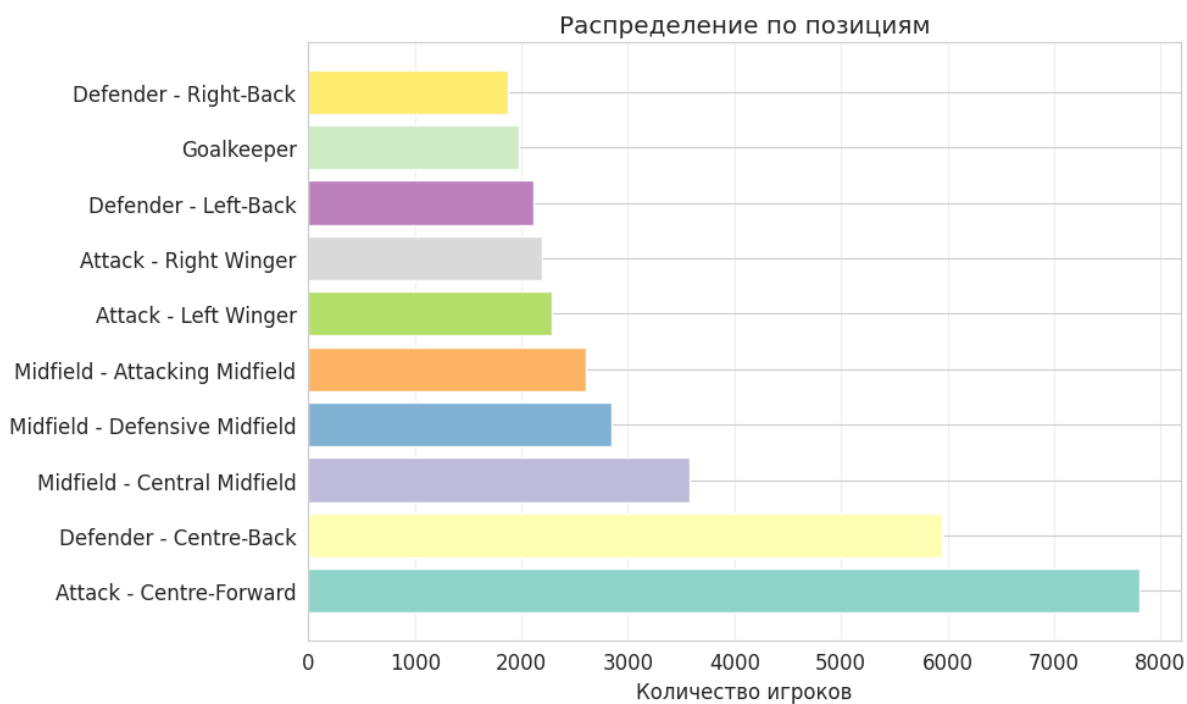


Рисунок 2.3 — Распределение игроков по позициям

На Рисунке 2.4 представлено распределение трансферных сумм. Гистограмма имеет правостороннюю асимметрию, большинство наблюдений сконцентрировано в области малых значений, основная масса трансферных сделок совершается на суммы до 10 миллионов евро. Дорогие трансферы (свыше 30 миллионов евро) являются редкими. После применения формулы (2.1), на Рисунке 2.5 представлено логарифмическое преобразование сумм трансферов, которое позволяет привести распределение к виду, близкому к нормальному, что улучшает сходимость алгоритмов обучения и уменьшает влияние выбросов.

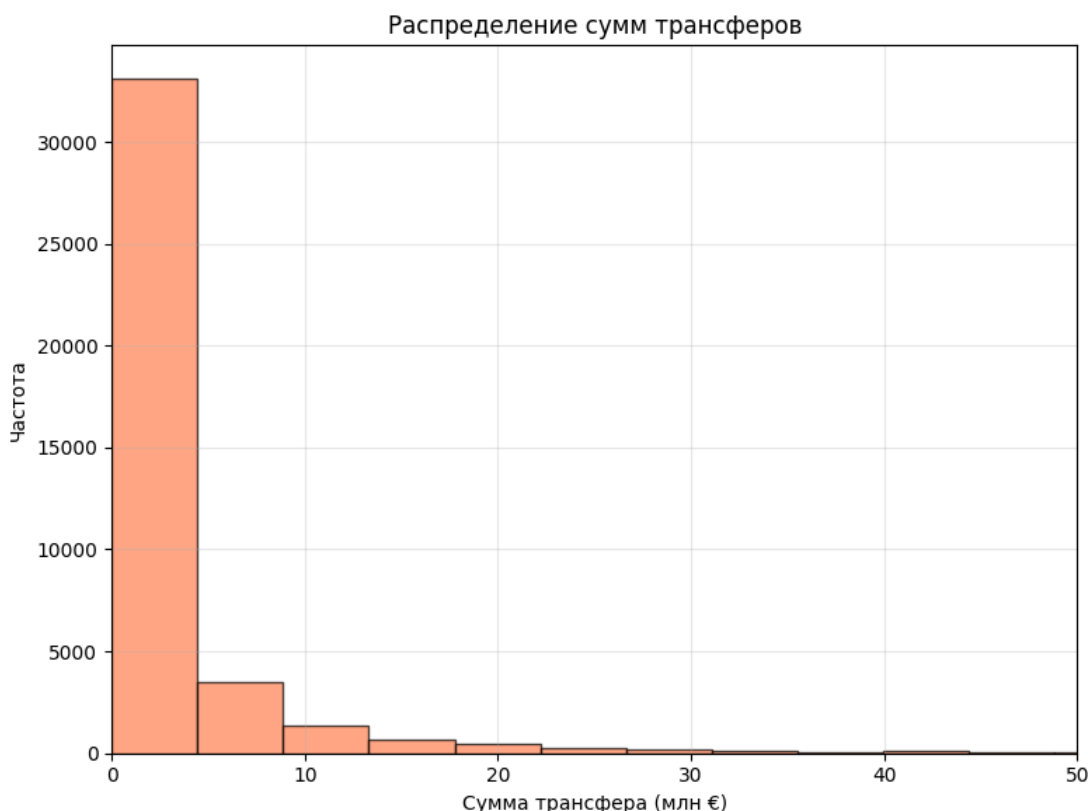


Рисунок 2.4 — Распределение трансферных сумм

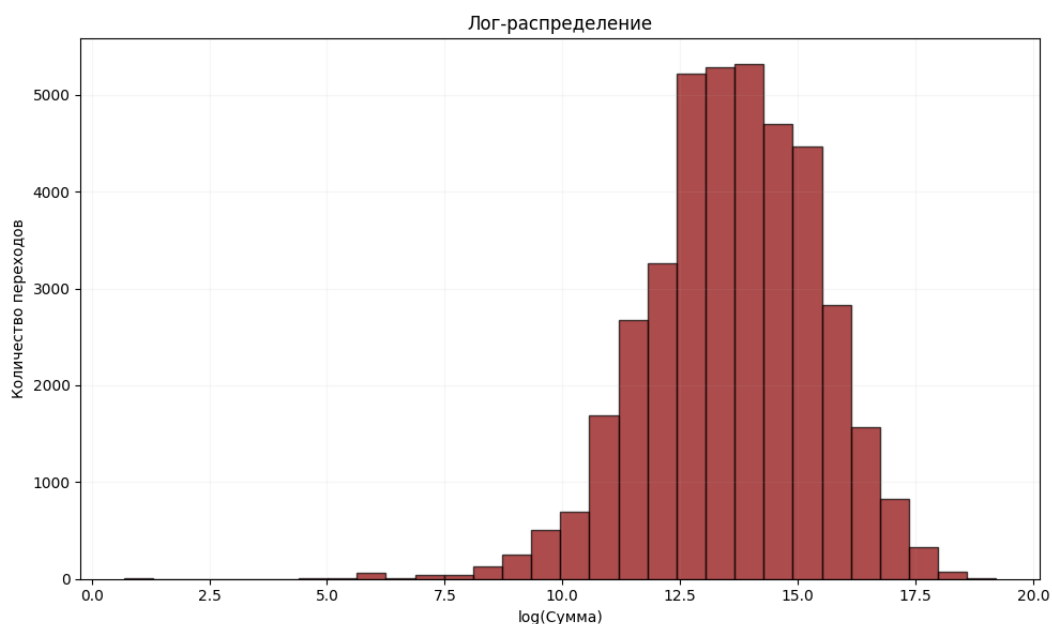


Рисунок 2.5 — Распределение трансферных сумм после логарифмирования

Зависимость средней трансферной стоимости от возраста игрока подтверждает экономическую логику рынка, что представлено на Рисунке 2.6. Цена растет по мере накопления опыта, достигая максимума к 26–28 годам, после чего наблюдается снижение средней стоимости, что обусловлено естественным ухудшением физических и игровых показателей.

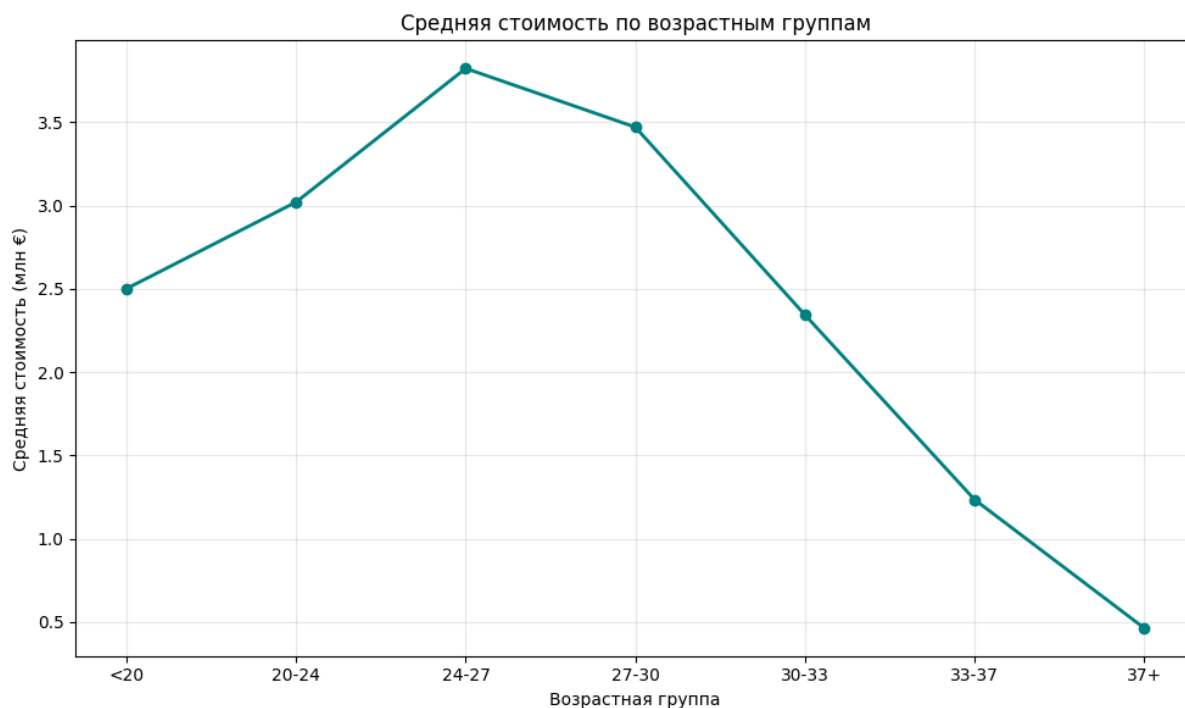


Рисунок 2.6 — Средняя стоимость по возрастным группам

На Рисунке 2.7 представлено изменение средней и медианной стоимости трансферов по сезонам, а также количество совершенных сделок. Наблюдается рост трансферных сумм, что свидетельствует об инфляции на рынке и требует учета временного фактора при построении прогностических моделей. Особый интерес представляет резкое сокращение трансферной активности в 2019–2021 годах, из-за пандемии COVID-19. В этот период наблюдалось снижение количества трансферов и средней стоимости сделок. Хотя рынок смог восстановиться после пандемии, количество сделок увеличилось, и цены на игроков продолжают расти.

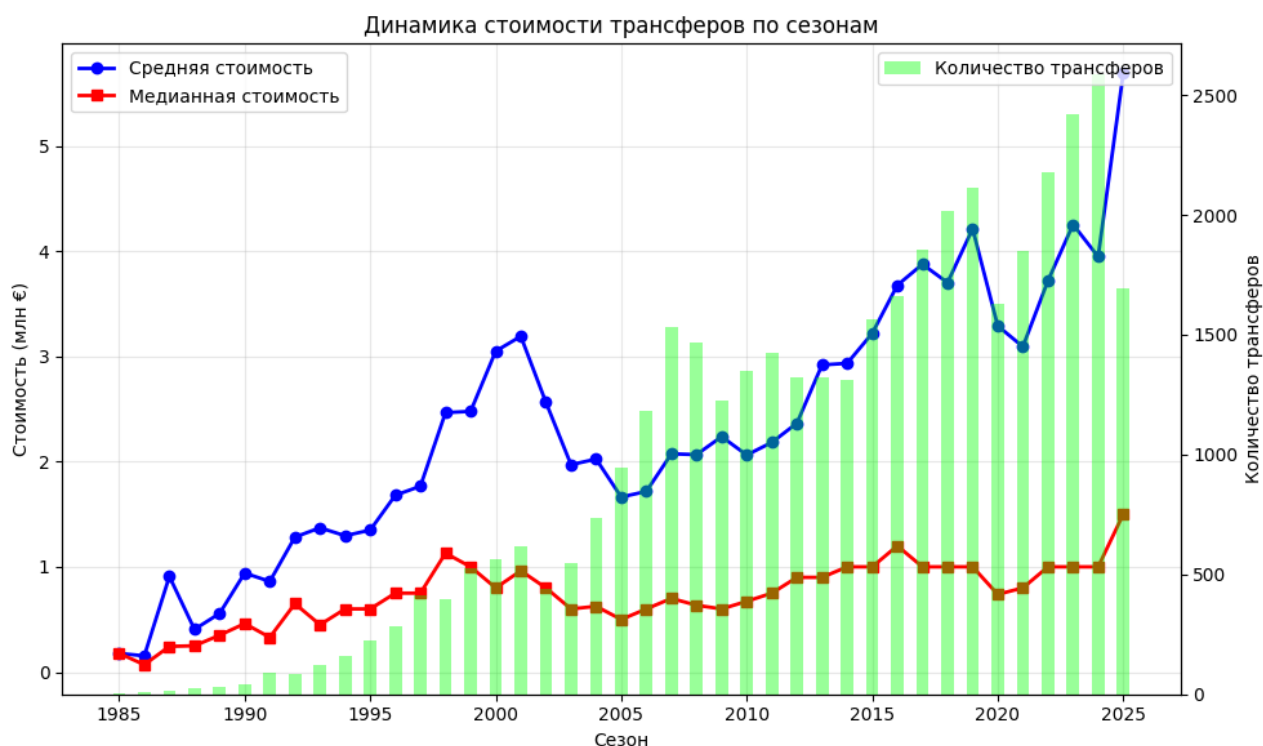


Рисунок 2.7 — Динамика стоимости трансферов по сезонам

В результате проведенного исследовательского анализа входных данных можно сделать вывод об их репрезентативности и пригодности для построения модели.

2.2 Описание этапов разработки модели

После подготовки и анализа данных приступаем к построению модели. Этот процесс включает несколько этапов, начиная от проектирования архитектуры решения и заканчивая выбором математических методов.

2.2.1 Проектирование архитектуры решения

Архитектура решения построена по модульному принципу и включает следующие основные модули, образующие единый конвейер (pipeline) обработки данных. Диаграмма компонентов системы показана на Рисунке 2.8.

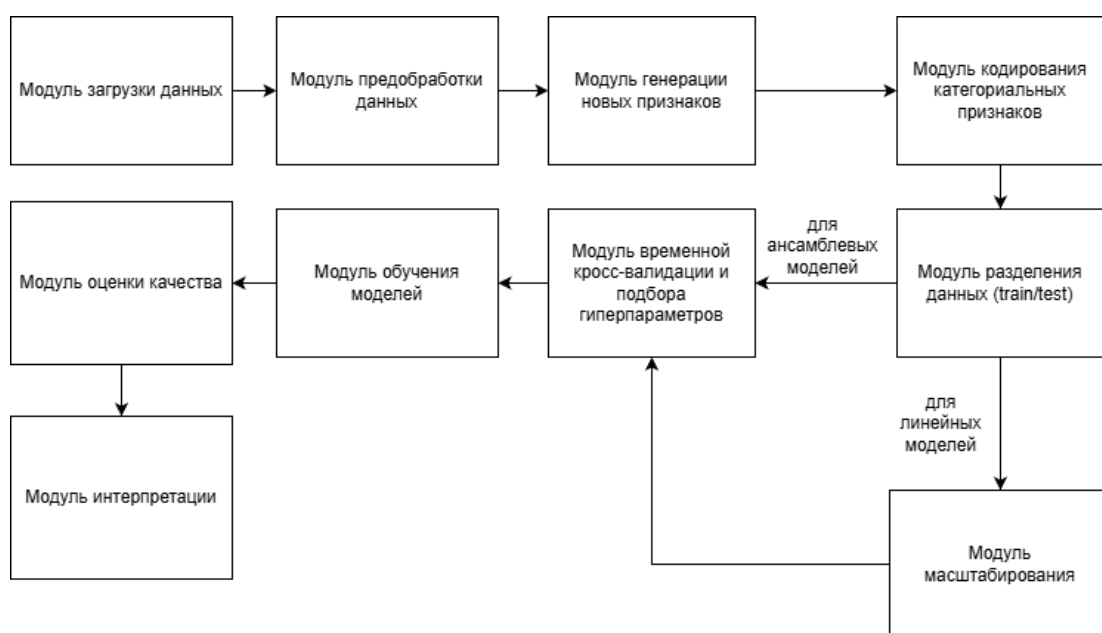


Рисунок 2.8 — Диаграмма компонентов системы

Взаимодействие модулей реализовано через вызов функций, передающих и преобразующих структуры данных `pandas.DataFrame`. Все модули вызываются последовательно, что обеспечивает воспроизводимость.

2.2.2 Описание математического аппарата реализуемого решения

Для решения задачи используются алгоритмы машинного обучения. В качестве базовой модели используется Ridge-регрессия (гребневая регрессия) с L2-регуляризацией [2.8]. Обычная линейная регрессия пытается подобрать коэффициенты так, чтобы ошибка была минимальна. Но на практике она легко переобучается, особенно если признаков много. Гребневая регрессия добавляет к функции потерь штраф за большие коэффициенты:

$$L(\beta) = \sum_{i=1}^n (y_i - X_i \beta)^2 + \alpha \sum_{j=1}^p \beta_j^2, \quad (2.5)$$

где y_i — фактическое значение целевой переменной,

X_i — вектор признаков,

β — вектор коэффициентов модели,

α — параметр регуляризации, контролирующий силу штрафа за большие коэффициенты.

Lasso представляет собой линейную регрессию с L1-регуляризацией. В отличие от гребневой регрессии, использующей квадратичный штраф, Lasso добавляет к функции потерь модуль штрафа. Особенностью L1-регуляризации является обнуление коэффициентов незначимых признаков:

$$L(\beta) = \sum_{i=1}^n (y_i - X_i \beta)^2 + \alpha \sum_{j=1}^p \|\beta_j\|, \quad (2.6)$$

Случайный лес (Random Forest) строит много решающих деревьев, каждое дерево строится на своей случайной подвыборке данных и использует случайное подмножество признаков для разделения узлов, а потом усредняет их предсказания. Это снижает переобучение и позволяет ловить более сложные зависимости.

$$\widehat{f(x)} = \frac{1}{M} \sum_{m=1}^M T_m(x), \quad (2.7)$$

где M — количество деревьев, $T_m(x)$ — прогноз m -го дерева.

Градиентный бустинг (Gradient Boosting) строит деревья последовательно, и каждое новое дерево пытается исправить ошибки предыдущего. В отличие от случайного леса, где деревья строятся независимо, в градиентном бустинге

каждое новое дерево обучается на остатках (разности между фактическими значениями и прогнозом текущего ансамбля).

$$F_m(x) = F_{m-1}(x) + \eta h_m(x), \quad (2.8)$$

где F_m — модель на шаге m , h_m — базовое дерево, обучаемое на отрицательном градиенте функции потерь, η — скорость обучения (learning rate) [2.9].

Ключевым математическим решением стала логарифмическая трансформация целевой переменной по формуле (2.1), это действие сжимает дорогие переходы, позволяя алгоритмам минимизировать относительную ошибку. Применение данных методов для оценки трансферной стоимости подтверждается современными исследованиями [2.10, 2.11]. В результате сравнительного анализа наилучшие показатели качества продемонстрировал случайный лес (Random Forest), который и был в качестве итоговой модели.

2.2.3 Разработка вычислительной схемы

Для объективной оценки моделей и подбора гиперпараметров применялась временная кросс-валидации TimeSeriesSplit. Этот метод гарантирует, что обучающая выборка всегда строго предшествует тестовой во времени, что соответствует реальной задаче прогнозирования будущих на основе исторических данных.

Поверх валидационной схемы был развернут алгоритм полного перебора гиперпараметров по заданной сетке (GridSearchCV). Вычислительная матрица формировалась следующим образом:

- Ridge: $\alpha \in \{1, 10, 50\}$;
- Lasso: $\alpha \in \{0.001, 0.01, 0.1\}$;
- Random Forest: количество деревьев (`n_estimators`) $\in \{100; 200\}$, максимальная глубина (`max_depth`) $\in \{15; 20\}$, минимальное число объектов в листе (`min_samples_split`) $\in \{5; 10\}$;

- Gradient Boosting: $n_estimators \in \{100; 200\}$, $learning_rate \in \{0,01; 0,05\}$, $max_depth \in \{3; 5\}$;

Оптимизация параметров выполнялась по метрике R^2 . Итоговое тестирование проводилось на отложенном временном отрезке (трансферы с 2022 по 2026 год), что гарантирует способность моделей прогнозировать будущие рыночные колебания.

2.3 Обоснование выбора инструментальных средств разработки

Реализация разработанной модели требует выбора инструментальных средств, от которых зависит не только удобство разработки, но и эффективность обработки данных и обучения модели. Для успешной реализации модели оценки стоимости футбольных трансферов был проведен сравнительный анализ существующих программных инструментов.

2.3.1 Сравнительный анализ и критерии выбора программных инструментов

Для выбора средств разработки необходимо учитывать: обработку больших данных, реализацию моделей, визуализацию.

При выборе рассматривались три инструмента: Python, R и C++.

Главные критерии выбора были такие:

- наличие библиотек;
- воспроизводимость результатов;
- производительность;
- работа с большими данными.

Результаты сравнения представлены в таблице 2.2.

Таблица 2.2 — Сравнительный анализ Python, R и C++

Критерий	Python	R	C++
Наличие библиотек для анализа данных	Pandas, numpy	dplyr, tidyr, data.table	используются внешние библиотеки и стандартные
Наличие библиотек для машинного обучения	Scikit-learn, Tensorflow, Pytorch, XGBoost, LightGBM, CatBoost	randomForest, caret, keras, xgboost	Shark, MLPACK, Dlib
Воспроизводимость результатов	Jupyter Notebook, Google Colab, VS Code	R Notebook, R Markdown	Visual Studio
Производительность	Высокая (оптимизированные библиотеки на C/C++)	Достаточная для статических расчетов	Высокая
Работа с большими данными	Хорошая благодаря библиотеке Spark	Ограниченные возможности, ориентирован на данные, помещающиеся в оперативную память	Хорошая работа

R хорош для статистики, но имеет ограниченные возможности для интеграции в веб-сервисы.

C++ обеспечивает максимальную производительность благодаря компиляции. Он эффективен для обработки больших данных и сложных вычислений, но у него сложный синтаксис, и мало библиотек для машинного обучения.

Python предлагает большой выбор библиотек для анализа данных и машинного обучения, обеспечивает высокую производительность благодаря языкам программированиям C/C++. Наличие таких инструментов, как Jupyter Notebook позволяет сочетать код, визуализацию, текст, что удобно для разработки модели.

Таким образом, для реализации модели был выбран Python, который обладает оптимальным балансом между скоростью разработки, читаемостью кода и наличием готовых библиотек для машинного обучения.

2.3.2 Описание выбранной среды разработки и требований к инфраструктуре

Основной средой разработки является Jupyter Notebook (в облачной платформе Google Colaboratory), которая позволяет сочетать код, визуализации, текст и формулы LaTeX в одном документе, что критично для разведочного анализа, прототипирования и документирования эксперимента. Облачная среда Google Colaboratory применялась для начальных экспериментов, так как она предоставляет бесплатный доступ к GPU/CPU и предустановленные библиотеки.

Для реализации модели использовались следующие библиотеки Python:

- pandas — для обработки табличных данных;
- numpy — для математических вычислений [2.12];
- scikit-learn — для реализации алгоритмов машинного обучения, предобработки данных и расчета метрик качества;
- matplotlib — для визуализации результатов.
- kagglehub — для автоматизированного скачивания актуальных датасетов.

Требования к инфраструктуре: оперативная память не менее 12 ГБ, многоядерный процессор для распараллеливания обучения случайного леса.

Выбранные технологии обеспечивают необходимый функционал для решения поставленной задачи: от загрузки и предобработки данных до обучения моделей и визуализации. Использование этих библиотек позволит эффективно реализовать все этапы разработки модели.

3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

3.1 Описание реализации модели

Данный этап включает программную реализацию всех ранее описанных компонентов и их интеграцию в единую систему.

Для более детальной реализации необходимо поэтапно описать процесс и взаимодействие компонентов.

3.1.1 Описание хода реализации

Реализация модели выполнена на языке программирования Python в облачной среде Google Colaboratory [3.1]. Процесс реализации включает следующие этапы:

1. Загрузка и обработка данных.

С помощью библиотеки `kagglehub` данные скачиваются автоматически (представлено в таблице 2.1). После скачивания загружаются несколько таблиц: история трансферов, профили игроков, рыночные стоимости, игровая статистика, травмы и выступления за сборную.

2. Предобработка данных.

На этапе предобработки из таблицы истории трансферов были исключены бесплатные переходы, поскольку их нулевая стоимость искажает алгоритмы ценообразования машинного обучения. Проблема отсутствующих данных решена методом статистической импутации: пропуски в числовых признаках заполнены медианным значением, а в категориальных — модой. Таблицы объединены в одну.

3. Формирование признаков.

Для построения прогнозной модели было сформировано 20 признаков, отражающих различные аспекты ценности игрока.

Для моделирования стоимости критически важен возраст игрока на момент совершения трансфера. Он рассчитывался как разность между датой трансфера и датой рождения игрока (с точностью до дня, затем перевод в годы). Поскольку зависимость рыночной стоимости от возраста нелинейна (стоимость быстро растёт у молодых игроков, достигает пика к 26–28 годам и постепенно снижается после 30 лет) в набор признаков был добавлен квадрат возраста. Это позволяет алгоритмам машинного обучения (особенно линейным моделям) учитывать параболическую динамику изменения стоимости на разных этапах карьеры игрока. На Рисунок 2.6 — Средняя стоимость по возрастным группам представлена средняя стоимость трансфера в зависимости от возрастной группы, что наглядно подтверждает описанный параболический тренд.

4. Синхронизация временных рядов [3.2].

Рыночная стоимость присоединена к основному массиву данных с помощью функции `pd.merge_asof`. Поскольку даты обновления рыночной стоимости и даты фактических трансферов редко совпадают, использовался поиск ближайшей предшествующей оценки в пределах 365 дней. Параметр `direction = «backward»` обеспечивает выбор оценки, которая была актуальна непосредственно перед трансфером. Для записей, у которых такая оценка отсутствовала, пропуски заполнялись медианным значением, но с учетом позиции и возрастной группы игрока. Это позволило сохранить все наблюдения и повысить репрезентативность данных.

5. Агрегация игровой статистики.

Для каждого трансфера собирались показатели игрока за два полных сезона до даты перехода, вычислялись метрики по формулам (2.2–2.4) [3.3]. Интегрированы данные о травмах (суммарное количество пропущенных дней до момента совершения трансфера), этот признак отражает физическую надежность игрока. По выступлениям за сборную для каждого игрока агрегированы голы,

забитые за национальную команду (суммарно за всю карьеру). Данные о матчах за сборную также были добавлены, но их влияние оказалось менее значимым.

6. Разделение данных по времени и масштабирование.

Данные разделены по временному принципу: обучающая выборка — трансферы с 1973 по 2021 год (30 447 записей), тестовая — с 2022 по 2026 год (9 550 записей). Для корректной работы алгоритмов, чувствительных к масштабу переменных (линейные модели), выполнено масштабирование числовых признаков с помощью StandardScaler. Этот метод трансформирует данные так, чтобы их среднее значение было равно нулю, а стандартное отклонение — единице. Древовидные ансамбли обучались на немасштабированных данных, так как они инвариантны к монотонным преобразованиям признакового пространства.

7. Подбор гиперпараметров.

Для каждого алгоритма задана сетка параметров. GridSearchCV выполнил перебор с TimeSeriesSplit. Оптимальные гиперпараметры каждой модели приведены в таблице 3.1.

Таблица 3.1 — Оптимальные гиперпараметры

Модель	Описание
Ridge (L2-регуляризация)	$\alpha = 50$, max_iter = 1000
Lasso (L1-регуляризация)	$\alpha = 0,01$, max_iter = 1000
Random Forest	n_estimators = 200, max_depth = 15, min_samples_split = 10
Gradient Boosting	n_estimators = 100, learning_rate = 0,05, max_depth = 5, subsample=0,8

8. Обучение моделей.

- Ridge (L2-регуляризация) добавляет квадратичный штраф на коэффициенты, снижая переобучение. Параметр регуляризации $\alpha=50$ подобран: при меньших значениях модель переобучалась, при больших — недообучалась (формула 2.5);
- Lasso (L1-регуляризация) использует модульный штраф, способный обнулять незначимые коэффициенты, выполняя отбор признаков. $\alpha=0,01$ — небольшое значение, позволяющее сохранить большинство признаков, но всё же оказывающее регуляризирующий эффект (формула 2.6);

- Random Forest строит 200 деревьев решений, каждое дерево строится на случайной подвыборке данных и случайном подмножестве признаков, а итоговый прогноз получается усреднением, максимальная глубина 15 ограничивает глубину дерева, предотвращая переобучение на шумах (формула 2.7) [3.4];
- Gradient Boosting последовательно строит деревья, каждое следующее исправляет ошибки предыдущего. Скорость обучения 0,05, модель строит 100 деревьев. subsample = 0,8 означает, что каждое дерево обучается на 80 % случайно выбранных данных, что дополнительно предотвращает переобучение (формула 2.8) [3.5].

Процесс обучения показал, что линейные модели обучились быстро (меньше чем за 100 итераций), что объясняется выпуклостью целевой функции и небольшим количеством признаков. Однако их итоговая предсказательная способность на тестовой выборке оказалась отрицательной ($R^2 < 0$) для Ridge и для Lasso. Это свидетельствует о том, что зависимость между характеристиками игрока и его трансферной стоимостью носит нелинейный характер, который линейные модели не могут подобрать.

Построение ансамблевых методов потребовало большего вычислительного времени, но в результате ансамблевые методы показали высокие метрики. Случайный лес достиг $R^2 = 0,7808$. Градиентный бустинг показал чуть низкий результат $R^2 = 0,7632$. Случайный лес можно считать наилучшей моделью.

3.1.2 Реализация интерфейсов и взаимодействия компонентов

Разработанная система не требует создания графического интерфейса (GUI), поскольку представляет собой аналитический конвейер, предназначенный для работы с данными. Взаимодействие компонентов системы

осуществляется посредством передачи и трансформации структур данных `pandas.DataFrame`.

Основные функции:

- `load_csv(file_path, description)` — загрузка CSV-файлов;
- `extract_season_year(season)` — извлечение года из названия сезона (например, обрезку строковых значений вида «2020/21» до временного формата 2020);
- `get_age_group(age)` — группировка возраста в категории;
- `calculate_advanced_metrics(df)` — расчёт продвинутых метрик;
- `find_player(player_name)` — поиск игрока по имени в профилях;
- `analyze_player_transfer(player_id, target_year)` — прогнозирование стоимости определенного игрока.

Взаимодействие компонентов реализовано через вызов функций из основного скрипта. На вход принимает `DataFrame` и возвращает преобразованный `DataFrame`. Для воспроизводимости результатов фиксирован `random_state = 42`. Полный листинг кода приведён в приложении Б.

3.2 Анализ полученных результатов

После завершения реализации модели необходимо оценить её качество. Это проверка точности прогнозов, устойчивости модели и её способности обобщать данные.

3.2.1 Описание и реализация методологии тестирования и валидации

Во избежание утечки будущей информации использована временная валидация. Обучающая выборка включала трансферы до 2021 года (1973–2021) более 30 тыс. записей, тестовая — с 2022 по 2026 год 9550 записей [3.6, 3.7]. При

этом учитывались подходы к обнаружению недооценённых игроков на основе рыночной динамики [3.8].

После обучения моделей был проведен сравнительный анализ их качества. Для оценки использовались следующие метрики:

- средняя абсолютная ошибка (MAE) — средняя абсолютная ошибка в евро. Показывает, на сколько в среднем прогноз отклоняется от фактической стоимости. Преимущество MAE — устойчивость к выбросам (формула 1.5);
- коэффициент детерминации (R^2) — доля дисперсии целевой переменной, объяснённая моделью. Значение $R^2 > 0$ означает, что модель даёт информацию о разбросе цен; $R^2 = 1$ соответствует идеальному прогнозу (формула 1.6)
- средняя абсолютная процентная ошибка (MAPE) — вычисляется как среднее абсолютных относительных отклонений, умноженное на 100 %. Позволяет оценить точность в процентах и удобна для сравнения с другими исследованиями (формула 1.7).

Все метрики вычислялись на тестовой выборке с использованием библиотеки `sklearn.metrics`.

Для контроля переобучения были рассчитаны метрики на обучающей выборке и на тестовой выборке представлено в таблице 3.2.

Таблица 3.2 — Метрики на обучающей и тестовой выборках (Random Forest)

Метрика	Обучающая выборка	Тестовая выборка
MAE	1 101 130 евро	1 772 882 евро
R^2	0,8276	0,7808
MAPE	7,5 %	42,5 %

Разница менее 0,05 по R^2 свидетельствует об отсутствии переобучения. Высокая MAPE на тесте объясняется наличием множества дешёвых трансферов (до 0,5 млн евро), где даже небольшая абсолютная ошибка даёт большой процент. Однако для практических целей важнее абсолютная ошибка — около 1,77 млн евро в среднем, что приемлемо для диапазона цен от 0 до 120 млн евро.

3.2.2 Сравнительный анализ и оценка эффективности с существующими решениями

Сравнение метрик между собой представлено на Рисунке 3.1.

Обучение: Ridge (L2)
Train: MAE = €2,338,962, $R^2 = -14.8802$, MAPE = 29.3%
Test : MAE = €3,103,832, $R^2 = -14.3914$, MAPE = 41.8%

Обучение: Lasso (L1)
Train: MAE = €2,239,226, $R^2 = -11.5699$, MAPE = 32.3%
Test : MAE = €2,864,391, $R^2 = -8.1105$, MAPE = 39.7%

Обучение: Random Forest
Train: MAE = €1,101,130, $R^2 = 0.8276$, MAPE = 7.5%
Test : MAE = €1,772,882, $R^2 = 0.7808$, MAPE = 42.5%

Обучение: Gradient Boosting
Train: MAE = €1,423,874, $R^2 = 0.7059$, MAPE = 13.7%
Test : MAE = €1,787,537, $R^2 = 0.7632$, MAPE = 49.3%

Лучшая модель на тесте: Random Forest ($R^2 = 0.7808$)

Рисунок 3.1 — Сравнение метрик

Лучшие показатели продемонстрировал случайный лес ($R^2 = 0,7808$), показывая, что модель объясняет 78 % разброса трансферных сумм. При этом средняя абсолютная ошибка (MAE) составила около 1,77 млн евро, а средняя относительная ошибка (MAPE) остановилась на уровне 42,5 %. Это связано с большим количеством трансферов малой стоимости, где даже небольшая абсолютная ошибка даёт большой процент.

Линейные модели (Ridge и Lasso) показали отрицательные значения коэффициента R^2 . Это объясняется тем, что зависимость между характеристиками игрока и его стоимостью носит нелинейный характер, которую способны эффективно улавливать только ансамблевые методы. Например, совместное влияние возраста и травм (травма в молодости менее критична, чем в возрасте после 30 лет) автоматически учитывается деревьями решений.

Для интерпретации работы были проведены два анализа важности [3.9, 3.10]. Анализ встроенной важности случайного леса (feature_importance) показал, что главным предиктором является предшествующая логарифм рыночная стоимость (32,1 %) (представлено на Рисунке 3.2), а также её рыночная стоимость (27,6 %). Далее следуют: сыгранных за сборную матчей (10,7 %), жёлтые карточки (4,5 %), и сыгранные минуты (4,0 %). Интересно, что возрастные признаки оказались менее значимыми (в сумме около 3 %), что может объясняться тем, что их влияние уже учтено в рыночной стоимости.

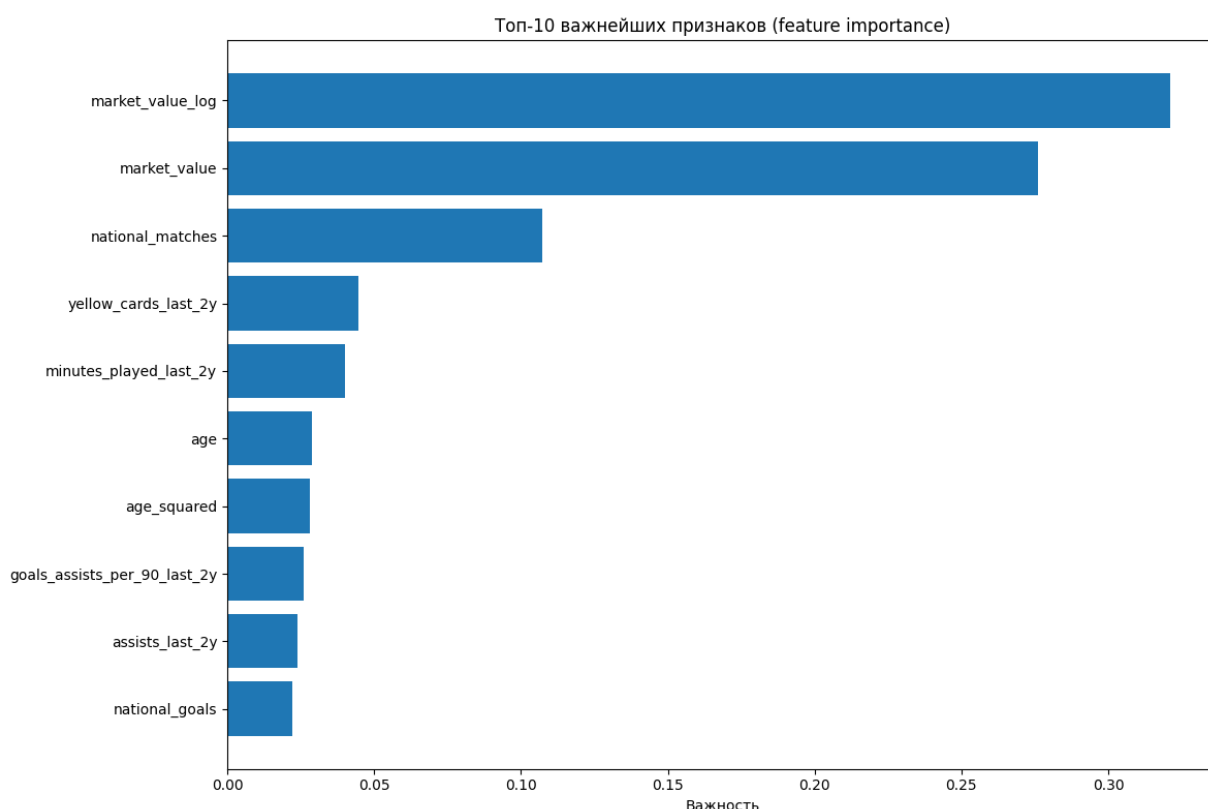


Рисунок 3.2 — Анализ важных признаков

Метод перестановочной важности (permutation importance) [3.11] подтвердил лидирующую позицию рыночной стоимости (представлено на Рисунке 3.3). Кроме того, данный метод вывел в топ значимых факторов возраст и квадрат возраста. Это математически доказывает изначальную гипотезу о высокой важности нелинейной зависимости в связке «возраст — стоимость», которую алгоритм использует для корректировки базовой оценки игрока.

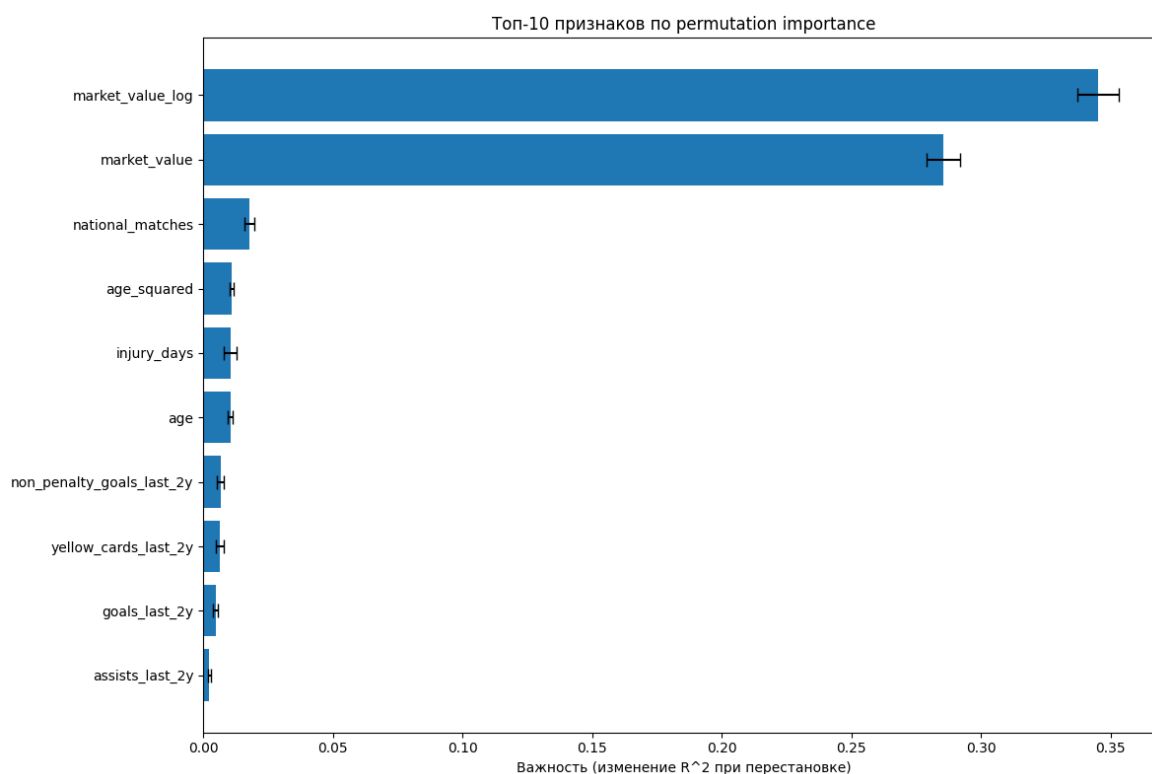


Рисунок 3.3 — Анализ важных признаков по permutation

В таблице 3.3 приведено сравнение ключевых характеристик разработанной модели на основе случайного леса с Transfermarkt и CIES.

Таблица 3.3 — Сравнение характеристик разработанной модели с Transfermarkt и CIES

Критерий	Transfermarkt	CIES	Разработанная модель
Тип подхода	Экспертный (коллективные оценки)	Статистический (линейная регрессия)	Random Forest (Машинное обучение)
Учёт нелинейности	Частично (опыт экспертов)	Нет (линейная модель)	Да (древовидный ансамбль)
Коэффициент детерминации	Не применим (нет единой)	~0,85	0,78
Обновление оценок	С задержкой	Автоматическое	Автоматическое
Требования к данным	Низкие	Средние	Низкие
Интерпретируемость	Человеческое объяснение (высокая)	Коэффициенты (средняя)	Важность признаков (высокая)

Модель CIES, основанная на линейной регрессии, по данным исследований демонстрирует R^2 около 0,85 на сопоставимых выборках. Предложенный Random Forest достигает $R^2=0,78$, при этом наша модель не требует ручного обновления и учитывает нелинейные зависимости, такие как взаимодействие возраста и травм. Transfermarkt, будучи экспертной платформой, не позволяет вычислить количественную метрику точности, однако страдает от субъективности и задержек в обновлении оценок.

Для проверки практической применимости модели были проанализированы трансферы известных футболистов. Относительная ошибка прогноза для каждого конкретного трансфера рассчитывалась по формуле:

$$\text{Ошибка} = \left(\frac{|\text{факт} - \text{прогноз}|}{\text{факт}} \right) * 100 \%. \quad (3.1)$$

где факт — фактическая стоимость трансфера, прогноз — предсказанное значение моделью. Данная метрика показывает, на сколько процентов прогноз приближен к реальной сумме.

Был проведен анализ шести известных футболистов (Бензема, Ибрагимович, Гризманн, Суарес, Аслани, Арнау Тенас). Модель показала относительные ошибки от 4,3 % до 15,3 %, что подтверждает практическую применимость модели.

Результаты сравнения представлены в таблице 3.4.

Таблица 3.4 — Результаты прогноза для конкретных трансферов

Игрок	Сезон	Фактическая стоимость	Прогноз	Ошибка
Карим Бензема	2009	35 млн. евро	31,9 млн. евро	8,7 %
Златан Ибрагимович	2011	24 млн. евро	22,7 млн. евро	5,4 %
Антуан Гризманн	2019	120 млн. евро	125,2 млн. евро	4,3 %
Луис Суарес	2022	10 млн. евро	9,4 млн. евро	5,7 %
Кристъян Аслани	2023	15,7 млн. евро	13,3 млн. евро	15,3 %
Арнау Тенас	2025	2,5 млн. евро	2,9 млн. евро	14,2 %

Для каждого из шести рассмотренных трансферов был проведён анализ причин отклонения прогнозной стоимости от фактической.

- Кристиан Аслани — модель ошиблась на 15,3 %, недооценив игрока. Причина в том, что он молодой (21 год), маленькая игровая практика (1436 минут за два сезона). Рынок разглядел в нём потенциал и заплатил выше.
- Карим Бензема — модель снова оценила сдержанно, опираясь только на статистику, а клубы часто переплачивают за потенциал молодых футболистов.
- Златан Ибрагимович — модель ошиблась на 5,4%, поскольку трансфер происходил на фоне конфликта игрока с тренером в «Милане», он был дешевле рыночной цены. Такие субъективные факторы в данных отсутствуют.
- Антуан Гризманн — этот игрок на пике карьеры, у него стабильная статистика и не было серьёзных травм. Доказывает, что модель корректно оценивает дорогостоящий трансфер.
- Луис Суарес — модель слегка занизила стоимость из-за возраста. Несмотря на выдающуюся карьеру в прошлом у этого игрока сократилось игровое время в последние сезоны перед трансфером.
- Арнау Тенас — модель переоценила игрока. Это связано с маленьким объёмом данных по вратарям и их показателям (сухие матчи и отражённые удары).

Таким образом, разработанная модель демонстрирует высокую предсказательную точность и может служить инструментом для предварительной оценки трансферной стоимости. Её преимущества — учёт широкого набора признаков, автоматическое выявление нелинейных зависимостей и использование только открытых данных. Дальнейшее улучшение возможно за счёт включения более детальной информации о контрактах, популярности в соцсетях и экономических показателях клубов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы был проведен детальный анализ предметной области, связанной с оценкой стоимости футбольных трансферов. На основе изучения существующих решений и выявления проблем была разработана модель машинного обучения.

В рамках исследовательского раздела были определены ключевые характеристики объекта и предмета исследования. Выявлены ключевые факторы, влияющие на стоимость игроков: игровые показатели, физические параметры, контрактные условия и субъективные факторы. Выполнен обзор существующих решений, включая экспертные платформы, алгоритмические системы и методы машинного обучения (Random Forest, Gradient boosting). Определены критерии успешности, а также приняты необходимые допущения и ограничения.

В аналитическом разделе были решены задачи сбора и обработки данных. Сформирован итоговый датасет, описывающий игроков. В качестве инструментальных средств был выбран язык программирования Python.

В технологическом разделе была выполнена практическая реализация в среде Google Colaboratory. Реализован полный конвейер обработки данных. Внедрен принцип временного разделения выборки, что исключило риск переобучения. Сравнительное тестирование четырех алгоритмов машинного обучения доказало подавляющее превосходство ансамблевых методов. Алгоритм случайного леса продемонстрировал наилучшие метрики качества. С помощью анализа важности признаков было математически доказано определяющее влияние возраста и недавней игровой практики на итоговую стоимость игрока.

Практическая применимость модели подтверждена на реальных трансферах известных футболистов: относительная ошибка прогноза составила от 4,3 % до 15,3 %. Модель может служить инструментом для оценки

трансферной стоимости, позволяя клубам и аналитическим агентствам снижать финансовые риски и принимать более обоснованные решения.

Таким образом, цель работы — создание модели оценки стоимости футбольных трансферов на основе игровой активности и физических характеристик игроков — достигнута. Разработанное решение позволяет:

- автоматизировать оценку стоимости с учетом нелинейных зависимостей;
- использовать только открытые данные, что обеспечивает доступность подхода;
- интерпретировать полученные прогнозы за счёт анализа важности признаков.

Перспективы дальнейших исследований включают расширение набора признаков (учёт контрактных условий, активности в социальных сетях, продвинутых метрик xG и xA), применение более сложных архитектур (нейронные сети, ансамбли с подкреплением), а также интеграцию модели в системы поддержки принятия решений для футбольных клубов.

Все поставленные задачи были успешно выполнены.

In the course of the work, a detailed analysis of the subject area related to the valuation of the football transfers was carried out. Based on the study of existing solutions and identified problems, a machine learning model was developed.

Within the framework of the research section, the key characteristics of the object and the subject of the research were determined. The key factors influencing the value of players were identified: game performance indicators, physical parameters, contractual conditions and subjective factors. A review of existing solutions, including expert platforms, algorithmic systems and machine learning methods (Random Forest, Gradient Boosting), was carried out. Success criteria were defined, and the necessary assumptions and limitations were adopted.

In the analytical section, the tasks of data collection and processing were solved. A final dataset describing the players was formed. Python was chosen as the instrumental tool.

In the technological section, a practical implementation was carried out in the Google Colaboratory environment. A full data processing pipeline was implemented. The principle of time-based splitting of the sample was introduced, which eliminated the risk of overfitting. Comparative testing of four machine learning algorithms proved the overwhelming superiority of ensemble methods. The Random Forest algorithm demonstrated the best quality metrics. By analyzing the importance of features, the determining influence of age, previous market value, and recent game practice on the final value of a player was mathematically proven.

The practical applicability of the model was confirmed on real transfers of famous football players: the relative forecast error ranged from 4.3% to 15.3%. The model can serve as a tool for assessing transfer value, allowing clubs and analytical agencies to reduce financial risks and make more informed decisions.

Thus, the goal of the work — the creation of a model for estimating the value of football transfers based on the gaming activity and physical characteristics of players — has been achieved. The developed solution allows:

- automating valuation taking into account nonlinear dependencies;
- using only open data, which ensures the accessibility of the approach;
- interpreting the obtained forecasts through the analysis of feature importance.

Prospects for further research include expanding the set of features (accounting for contract terms, social media activity, advanced xG and xA metrics), applying more complex architectures (neural networks, ensembles with reinforcement), as well as integrating the model into decision support systems for football clubs.

All the tasks set were successfully completed.

СПИСОК ИСТОЧНИКОВ

1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

- 1.1 Rico-González, M. et al. Machine learning application in soccer: a systematic review // *Biology of Sport*. — 2023. — Vol. 40, № 1. — P. 243–263.
- 1.2 Franceschi M., Brocard J.-F., Follert F., Gougnet J.-J. Determinants of football players' valuation: A systematic review // *Journal of Economic Surveys*. — 2024. — Vol. 38, № 3. — P. 577–600.
- 1.3 Antonioni P., Cubbin J. The Bosman Case and Football Transfer Market Efficiency: 25 Years On // *Journal of Sports Economics*. — 2021. — Vol. 22, № 3. — P. 256–280.
- 1.4 Солнцев И.В., Осокин Н.А. Спортивные и экономические детерминанты трансферных сделок в российском футболе // *Финансы и бизнес*. — 2022. — № 2. — С. 80–95.
- 1.5 Егоров Н.А. Влияние пандемии COVID-19 на трансферную политику футбольных клубов Российской Премьер-Лиги // *Экономика и предпринимательство*. — 2022. — № 3. — С. 1154–1159.
- 1.6 Солнцев И.В. Экономические потери европейских футбольных клубов, вызванные коронавирусом // *Эко*. — 2022. — № 2. — С. 40–53.
- 1.7 Айбатуллин Т.А., Касимова А.Х. ИИ и трансферный рынок: автоматизация оценки стоимости и потенциала футболистов // *Инновации и инвестиции*. — 2024. — № 5. — С. 39–44.
- 1.8 Transfermarkt — Football Transfers Database [Электронный ресурс]. — URL: <https://www.transfermarkt.com> (дата обращения: 01.03.2026).
- 1.9 CIES Football Observatory — Player Transfer Values [Электронный ресурс]. — URL: <https://football-observatory.com> (дата обращения: 01.03.2026).
- 1.10 Poli R., Besson R., Ravenel L. Statistical Modeling of Football Players' Transfer Fees Worldwide // *International Journal of Financial Studies*. — 2024. — Vol. 12, № 3. — P. 93.

- 1.11 Коновалов М.С., Емельянова Т.В. Прогнозирование трансферной стоимости футболистов с использованием различных регрессионных метод и алгоритмов машинного обучения // Все грани математики и механики: сборник статей Всероссийской молодежной научной конференции. — Томск: STT, 2024. — С. 76–80.
- 1.12 Sulimov D. Performance Insights-based AI-driven Football Transfer Fee Prediction // arXiv preprint arXiv:2401.16795. – 2024.
- 1.13 Lee H., Tama B.A., Cha M. Prediction of Football Player Value using Bayesian Ensemble Approach // arXiv preprint arXiv:2206.13246. — 2022.
- 1.14 Григорян К.Г. Эконометрический подход к оценке трансферной стоимости футболиста // Весенние дни науки: сборник докладов Международной конференции студентов и молодых ученых. — Екатеринбург: УрФУ, 2023. — С. 1381–1383.

2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

- 2.1 Kaggle — Football Datasets [Электронный ресурс]. — 2024. URL: <https://www.kaggle.com/datasets/xfkzujqjvx97n/football-datasets/data> (дата обращения: 01.03.2026).
- 2.2 pandas documentation [Электронный ресурс]. — 2024. — URL: <https://pandas.pydata.org/docs/> (дата обращения: 01.03.2026).
- 2.3 James G., Witten D., Hastie T., Tibshirani R. An Introduction to Statistical Learning with Application in R. — 2nd ed.— New York: Springer, 2021. — 607 p.
- 2.4 Документация библиотеки Scikit-learn [Электронный ресурс]. — URL: https://scikit-learn.org/stable/user_guide.html (дата обращения: 01.03.2026).
- 2.5 Титов А.Н., Тазиева Р.Ф. Обработка данных в Python. Основы работы с библиотекой Pandas: учебно-методическое пособие. — Казань: КНИТУ, 2022. — 116 с.

- 2.6 Matplotlib Documentation [Электронный ресурс]. — URL: <https://matplotlib.org/stable/contents.html> (дата обращения: 01.03.2026).
- 2.7 Yalçinkaya M.A., Işık M. The Role of Performance Metrics in Estimating Market Values of Footballers in Europe's Top Five Leagues // Pamukkale Journal of Sport Sciences. — 2024. — Vol. 15, № 3. — P. 455–485.
- 2.8 VanderPlas J. Python Data Science Handbook. — Sebastopol: O'Reilly Media, 2023. — 548 p.
- 2.9 Géron A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. — 2nd ed. — O'Reilly Media, 2022. — 672 p.
- 2.10 van Arem K., Goes-Smit F., Söhl J. Forecasting the future development in quality and value of professional football players for applications in team management // arXiv preprint. — 2025. — arXiv:2502.07528.
- 2.11 Рашка С., Лю Ю., Мирджалили В. Машинное обучение с PyTorch и Scikit-Learn. — Астана: Фолиант, 2024. — 688 с.
- 2.12 NumPy Developer Guide [Электронный ресурс]. — URL: <https://numpy.org/doc/stable/> (дата обращения: 01.03.2026).

3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

- 3.1 Среда разработки Google Colaboratory [Электронный ресурс]. — URL: <https://colab.research.google.com/> (дата обращения: 29.03.2026).
- 3.2 Маккини У. Python и анализ данных: Data Wrangling with pandas, NumPy и Jupyter. — 3-е изд. — М.: ДМК Пресс, 2023. — 540 с.
- 3.3 Шолле Ф. Глубокое обучение на Python. — 2-е изд. — М.: ДМК Пресс, 2022. — 480 с.
- 3.4 Документация Scikit-learn: RandomForestRegressor [Электронный ресурс]. — URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html> (дата обращения: 01.03.2026).
- 3.5 Документация Scikit-learn: GradientBoostingRegressor [Электронный ресурс]. — URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html>

[learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html](https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html) (дата обращения: 01.03.2026).

3.6 Yang Y., Königstorfer J., Pawlowski T. Predicting transfer fees in professional European football before and during COVID-19 using machine learning // European Sport Management Quarterly. — 2024. — Vol. 24, № 3. — P. 603–623.

3.7 Molnar C. Interpretable Machine Learning [Электронный ресурс]. — 2023. — URL: <https://christophm.github.io/interpretable-ml-book/> (дата обращения: 01.03.2026).

3.8 Omejjeke C. et al. Objective Mispricing Detection for Shortlisting Undervalued Football Players via Market Dynamics and News Signals // arXiv preprint. — 2026. — arXiv:2603.17687.

3.9 Лемешко Б.Ю., Лемешко С.Б. Статический анализ данных, моделирование и исследование вероятностных закономерностей. Компьютерный подход. — М.: ИНФРА-М, 2023. — 327 с.

3.10 Воронцов К.В. Машинное обучение: курс лекций [Электронный ресурс]. — URL: <http://www.machinelearning.ru/wiki/> (дата обращения: 01.03.2026).

3.11 Scikit-learn: permutation importance [Электронный ресурс]. — URL: https://scikit-learn.org/stable/modules/permutation_importance.html (дата обращения: 01.03.2026).

ПРИЛОЖЕНИЯ

Приложение А — Графический материал (презентация)

Приложение Б — Пример реализации кода

Приложение А

Графический материал



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА - Российский технологический университет»

Институт информационных технологий (ИИТ)

Кафедра прикладной математики

Направление «Прикладная математика»

профиль «Анализ данных»

Выпускная квалификационная работа бакалавра на тему:

«Разработка модели оценки стоимости футбольных трансферов»

Автор: студент группы ИМБО-02-22

Ким Кирилл Сергеевич

Руководитель: к.ф.-м.н., доцент кафедры прикладной математики

Даева Софья Георгиевна

Москва 2026

Рисунок А.1 — Вступительный слайд

Цель, объект и предмет исследования

Актуальность: Объём трансферного рынка превышает 7 млрд евро в год. Решения о стоимости игроков принимаются на интуиции, что создаёт финансовые риски.

Цель: Разработать модель машинного обучения для прогнозирования стоимости футбольных трансферов на основе игровой активности и физических характеристик игроков.

Объект исследования — процесс формирования стоимости игроков на рынке трансферов.

Предмет исследования — математическая модель прогнозирования трансферной стоимости.

2

Рисунок А.2 — Цель работы

Задачи исследования

1. Изучить предметную область и ключевые факторы стоимости.
2. Провести обзор существующих решений.
3. Сформулировать задачу, критерии, ограничения.
4. Выполнить сбор, очистку и предобработку данных.
5. Спроектировать новые признаки.
6. Обучить модели с подбором гиперпараметров.
7. Провести сравнительный анализ.
8. Протестировать на реальных трансферах.

Критерии успешности:

- MAE (средняя абсолютная ошибка) $\approx 3,0$ млн евро;
- R^2 (коэффициент детерминации) $\geq 0,5$;
- $MAPE$ (средняя абсолютная процентная ошибка) $\leq 50\%$.

3

Рисунок А.3 — Задачи исследования

Характеристика предметной области

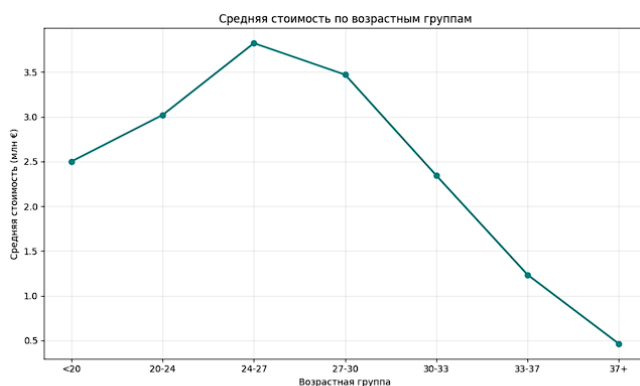


Рисунок 1 — Средняя стоимость по возрастным группам

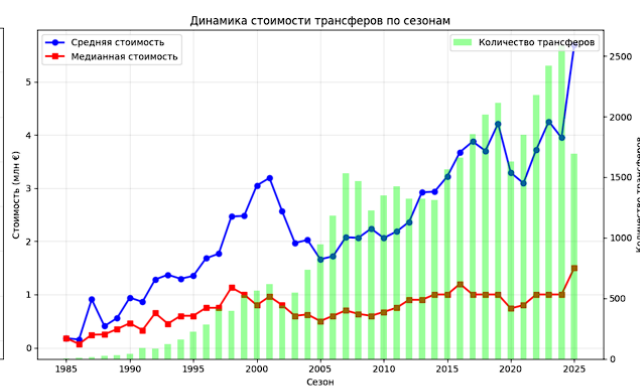


Рисунок 2 — Динамика стоимости трансферов по сезонам

Пик стоимости 26–29 лет.

Факторы стоимости: игровая статистика (голы, передачи), контракт, травмы, рыночные тренды, субъективные факторы.

4

Рисунок А.4 — Характеристики предметной области

Обзор существующих решений

Таблица 1 — Существующие решения

Решение	Подход	Недостатки
<u>Transfermarkt</u>	Коллективная экспертная оценка	Субъективность, редкие обновления, ошибки из-за симпатий к клубам.
Модели на основе статистики (CIES)	<u>Регрессионный анализ</u> (Используют коэффициенты прошлого опыта)	Не учитывает нелинейности, завышает топ-лиги
Машинное обучение	Random Forest, Gradient Boosting	Нужны большие данные, риск переобучения, настройки <u>гиперпараметров</u>

Transfermarkt субъективен и редок, CIES линеен и недоступен. Целесообразно использовать ML-методы.

5

Рисунок A.5 — Обзор существующих решений

Формулировка исследовательской задачи

Постановка задачи:

Обучающая выборка: $D = \{(x_i, y_i)\}_{i=1}^n$,

Целевая переменная логарифмируется: $y' = \ln(y + 1)$ — для нормализации,

Требуется построить $f: X \rightarrow Y$, минимизирующую MAE и максимизирующую R^2 .

Допущения: только открытые данные, только платные трансферы, временной сплит (обучение ≤ 2021 , тест ≥ 2022).

Ограничения: модель не учитывает ход переговоров и субъективные факторы.

6

Рисунок A.6 — Формулировка исследовательской задачи

Анализ и характеристика входных данных

Источник данных — Kaggle, датасет «Football Datasets» ([Transfermarkt](#)). 6 таблиц, объём >4 млн. записей.

Таблица 2 — Характеристики исходных таблиц

Название файла	Описание
transfer_history.csv	История трансферов игроков
player_profiles.csv	Профили игроков (дата рождения, позиция и др.)
player_market_value.csv	Динамика рыночной стоимости
player_performances.csv	Детальная игровая статистика по сезонам
player_injuries.csv	Данные о травмах
player_national_performances.csv	Статистика выступлений за сборную

7

Рисунок А.7 — Анализ входных данных

ER-диаграмма

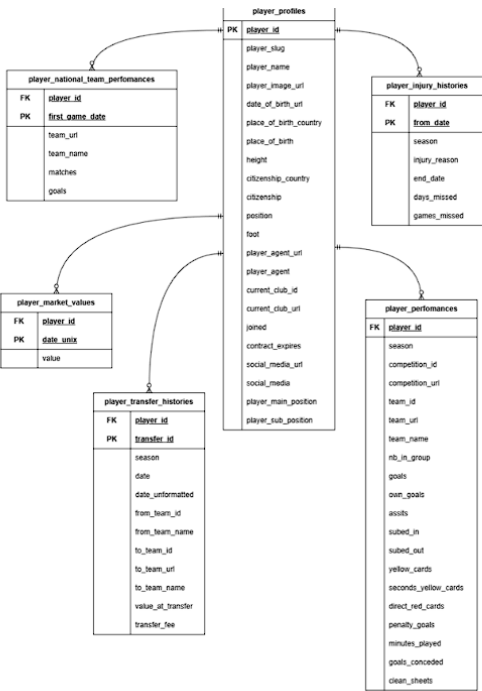
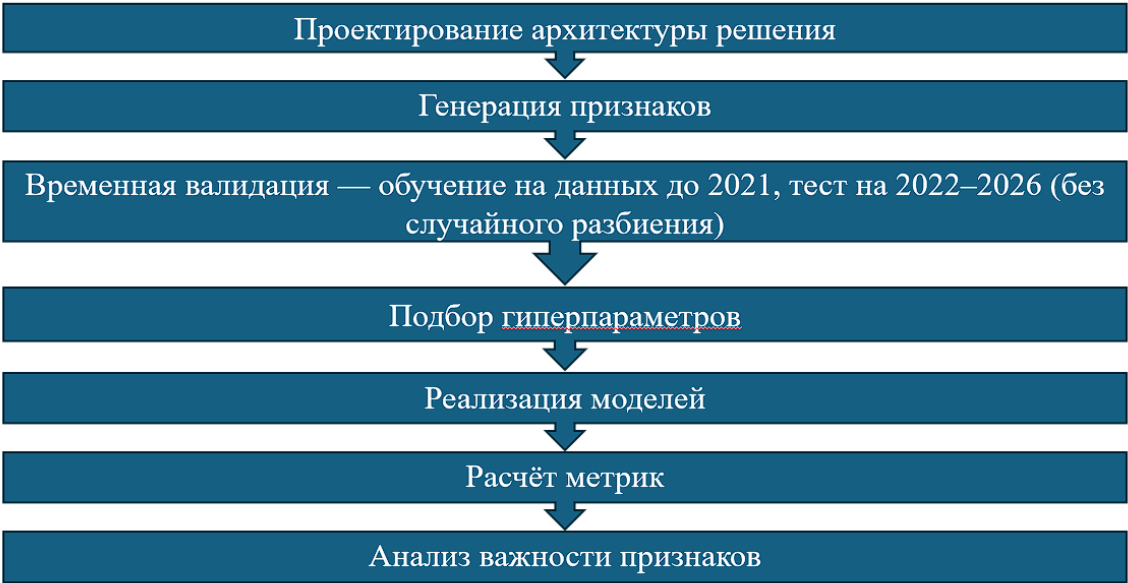


Рисунок 3— ER-диаграмма исходных таблиц

8

Рисунок А.8 — ER-диаграмма

Описание этапов разработки модели



10

Рисунок А.9 — Описание этапов разработки модели

Реализация модели и анализ результатов

Подбор через GridSearchCV с TimeSeriesSplit

Таблица 3 — Оптимальные гиперпараметры для всех моделей

Модель	Описание
<u>Ridge</u> (L2-регуляризация)	$\alpha = 50$
<u>Lasso</u> (L1-регуляризация)	$\alpha = 0,01$
Random Forest	200 деревьев, максимальная глубина 15
<u>Gradient Boosting</u>	100 деревьев, скорость обучения 0,05, глубина 5

```
Обучение: Ridge (L2)
Train: MAE = €2,338,962, R² = -14.8802, MAPE = 29.3%
Test : MAE = €3,103,832, R² = -14.3914, MAPE = 41.8%

Обучение: Lasso (L1)
Train: MAE = €2,239,226, R² = -11.5699, MAPE = 32.3%
Test : MAE = €2,864,391, R² = -8.1105, MAPE = 39.7%

Обучение: Random Forest
Train: MAE = €1,101,130, R² = 0.8276, MAPE = 7.5%
Test : MAE = €1,772,882, R² = 0.7808, MAPE = 42.5%

Обучение: Gradient Boosting
Train: MAE = €1,423,874, R² = 0.7059, MAPE = 13.7%
Test : MAE = €1,787,537, R² = 0.7632, MAPE = 49.3%

Лучшая модель на тесте: Random Forest (R² = 0.7808)
```

12

Рисунок 6 — Сравнение метрик

Рисунок А.10 — Реализация модели

Важность признаков

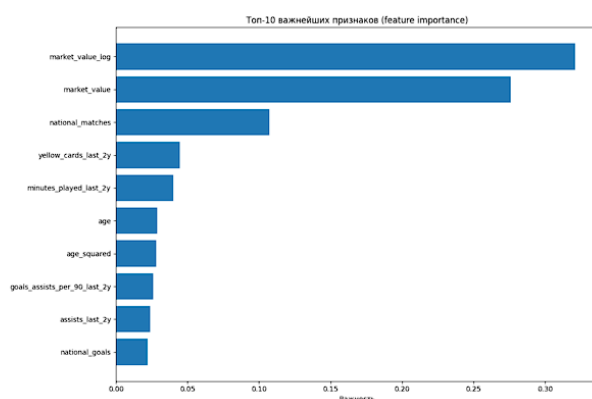


Рисунок 7 — Анализ важных признаков

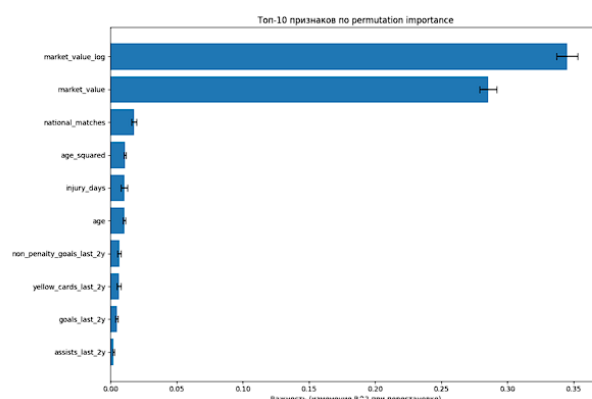


Рисунок 8 — Анализ важных признаков по permutation

Логарифм рыночная стоимость — главный фактор (32,1%).
Permutation importance подтверждает значимость возраст и возраст².

13

Рисунок A.11 — Анализ важности модели

Тестирование на реальных трансферах

Таблица 4 — Результаты прогноза для конкретных трансферов

Игрок	Сезон	Фактическая стоимость	Прогноз	Ошибка
Карим <u>Бензема</u>	2009	35 млн. евро	31,9 млн. Евро	8,7 %
Златан Ибрагимович	2011	24 млн. евро	22,7 млн. Евро	5,4 %
Антуан <u>Гризманн</u>	2019	120 млн. евро	125,2 млн. Евро	4,3 %
Луис Суарес	2022	10 млн. евро	9,4 млн. Евро	5,7 %
Кристиан <u>Аслани</u>	2023	15,7 млн. евро	13,3 млн. Евро	15,3 %
<u>Арнау Тенас</u>	2025	2,5 млн. евро	2,9 млн. евро	14,2 %

Относительная ошибка прогноза на трансферах (Бензема, Гризманн, Суарес и др.) составила от 4,3% до 15,3%, что подтверждает практическую применимость модели.

14

Рисунок A.12 — Тестирование на реальных трансферах

Заключение

Цель достигнута: разработана модель оценки стоимости футбольных трансферов;

Задачи решены:

- Проанализирована предметная область и выделены ключевые факторы стоимости.
- Проведён обзор существующих решений ([Transfermarkt](#), CIES).
- Сформулирована исследовательская задача, определены метрики и ограничения.
- Выполнены сбор, очистка, предобработка данных и генерация новых признаков.
- Обучены и сравнены 4 модели ML. Лучшая — [Random Forest](#) ($R^2 = 0,7808$, $MAE = 1,773$ млн €).
- Модель протестирована на реальных трансферах (ошибка 4,3% – 15,3%);

Подтверждена практическая применимость;

15

Рисунок А.13 — Заключение

Приложение Б

Пример реализации логики программы

Листинг Б.1 — Импорт библиотек и загрузка данных

```
!pip install kagglehub -q
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, r2_score,
mean_absolute_percentage_error
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.inspection import permutation_importance
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
import kagglehub
path = kagglehub.dataset_download("xfkzujqjvx97n/football-datasets")
def load_csv(file_path, description=""):
    full_path = os.path.join(path, file_path)
    df = pd.read_csv(full_path, low_memory=False)
    print(f"    {description:30}: {len(df):8,} rows, {len(df.columns):3d}
cols")
    return df
transfer_history = load_csv('transfer_history/transfer_history.csv',
"История трансферов")
player_profiles = load_csv('player_profiles/player_profiles.csv', "Профили
игроков")
market_values = load_csv('player_market_value/player_market_value.csv',
"Рыночная стоимость")
player_performances =
load_csv('player_performances/player_performances.csv', "Выступления игроков")
player_injuries = load_csv('player_injuries/player_injuries.csv', "Травмы
игроков")
player_national =
load_csv('player_national_performances/player_national_performances.csv',
"Сборная")
```


Листинг Б.2 — Вспомогательные функции (часть 1)

```
def extract_season_year(season):
    if pd.isna(season):
        return None
    s = str(season).strip()
    if '/' in s:
        first = s.split('/')[0]
        if len(first) == 4:
            return int(first)
        elif len(first) == 2:
            return 2000 + int(first) if int(first) < 50 else 1900 +
int(first)
    if s.isdigit() and len(s) == 4:
        return int(s)
    return None

def get_age_group(age):
    if pd.isna(age):
        return 'Unknown'
    if age < 20:
        return '<20'
    elif age < 24:
        return '20-24'
    elif age < 28:
        return '24-27'
    elif age < 31:
        return '27-30'
    elif age < 34:
        return '30-33'
    elif age < 37:
        return '33-37'
    else:
        return '37+'
```

Листинг Б.3 — Вспомогательные функции (часть 2)

```
def calculate_advanced_metrics(df_perf):
    df = df_perf.copy()
    if 'goals' in df.columns and 'assists' in df.columns and
'minutes_played' in df.columns:
        df['goals_assists_per_90'] = (df['goals'] + df['assists']) /
(df['minutes_played'] / 90 + 1e-5)
        df['goals_assists_per_90'] =
df['goals_assists_per_90'].replace([np.inf, -np.inf], 0).fillna(0)
    if 'penalty_goals' in df.columns and 'goals' in df.columns:
        df['non_penalty_goals'] = df['goals'] -
df['penalty_goals'].fillna(0)
    if 'clean_sheets' in df.columns and 'goals_conceded' in df.columns:
        df['clean_sheet_ratio'] = df['clean_sheets'] /
(df['goals_conceded'] + 1)
    return df
```

Листинг Б.4 — Очистка данных и расчёт возраста

```
df_transfers = transfer_history[transfer_history['transfer_fee'] >
0].copy()
df_transfers['log_fee'] = np.log1p(df_transfers['transfer_fee'])
df_transfers['season_year'] =
df_transfers['season_name'].apply(extract_season_year)
df_transfers = df_transfers.dropna(subset=['transfer_date', 'player_id'])
player_profiles['birth_date'] =
pd.to_datetime(player_profiles['date_of_birth'], errors='coerce')
df_transfers['transfer_date'] =
pd.to_datetime(df_transfers['transfer_date'], errors='coerce')
# Объединение с профилями
df = df_transfers.merge(player_profiles[['player_id', 'birth_date',
'position', 'height', 'foot']], on='player_id', how='left')
# Расчёт возраста
df['age'] = (df['transfer_date'] - df['birth_date']).dt.days / 365.25
df['age'] = df['age'].round(0)
age_median = df['age'].median()
df['age'] = df['age'].fillna(age_median)
df['age_squared'] = df['age'] ** 2
df['age_group'] = df['age'].apply(get_age_group)
```

Листинг Б.5 — Разведочный анализ (часть 1)

```
# Распределение по возрасту
plt.figure(figsize=(12,6))
age_bins = np.arange(15, 41, 2)
plt.hist(df['age'], bins=age_bins, color='blue', edgecolor='black',
alpha=0.7)
plt.axvline(x=df['age'].median(), color='red', linestyle='--',
linewidth=2,
label=f"Медиана: {df['age'].median():.1f}")
plt.xlabel('Возраст (лет)')
plt.ylabel('Количество игроков')
plt.title('Распределение игроков по возрасту')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Распределение по позициям
plt.figure(figsize=(10,6))
position_counts = df['position'].value_counts().head(10)
colors = plt.cm.Set3(np.linspace(0, 1, len(position_counts)))
plt.barh(position_counts.index, position_counts.values, color=colors)
plt.xlabel('Количество игроков')
plt.title('Распределение по позициям')
plt.grid(True, alpha=0.3, axis='x')
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(8, 6))
# Распределение стоимости трансферов
fee_millions = df['transfer_fee'] / 1_000_000
ax.hist(fee_millions, bins=50, color='coral', edgecolor='black',
alpha=0.7)
ax.set_xlabel('Сумма трансфера (млн €)')
ax.set_ylabel('Частота')
ax.set_title('Распределение сумм трансферов')
ax.set_xlim([0, 50])
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Листинг Б.6 — Разведочный анализ (часть 2)

```
fig, ax = plt.subplots(figsize=(10, 6))
# Логарифмическим распределением
ax.hist(np.log1p(df['transfer_fee']), bins=30, color='darkred',
edgecolor='black', alpha=0.7)
ax.set_xlabel('log(Сумма)')
ax.set_ylabel('Количество переходов')
ax.set_title('Лог-распределение')
ax.grid(True, alpha=0.1)
plt.tight_layout()
plt.show()

# Средняя стоимость по возрастным группам
age_groups = ['<20', '20-24', '24-27', '27-30', '30-33', '33-37', '37+']
df['age_group_simple'] = pd.cut(df['age'], bins=[0, 20, 24, 27, 30, 33, 37,
50], labels=age_groups)
age_group_avg = df.groupby('age_group_simple')['transfer_fee'].mean() /
1_000_000
plt.figure(figsize=(10, 6))
plt.plot(age_group_avg.index, age_group_avg.values, marker='o',
color='teal', linewidth=2)
plt.xlabel('Возрастная группа')
plt.ylabel('Средняя стоимость (млн €)')
plt.title('Средняя стоимость по возрастным группам')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Листинг Б.7 — Разведочный анализ (часть 3)

```
# Динамика по сезонам
season_stats = df.groupby('season_year').agg({'transfer_fee': ['mean',
'median', 'count']}).round(0)
season_stats.columns = ['mean_fee', 'median_fee', 'count']
season_stats = season_stats[(season_stats.index >= 1985) &
(season_stats.index <= 2025)]

fig, ax = plt.subplots(figsize=(10,6))
ax.plot(season_stats.index, season_stats['mean_fee'] / 1_000_000,
marker='o', linewidth=2, markersize=6, color='blue', label='Mean fee')
ax.plot(season_stats.index, season_stats['median_fee'] / 1_000_000,
marker='s', linewidth=2, markersize=6, color='red', label='Median fee')
ax.set_xlabel('Сезон')
ax.set_ylabel('Стоимость (млн €)')
ax.set_title('Динамика стоимости трансферов по сезонам')
ax.grid(True, alpha=0.3)
ax.legend(loc='upper left')

ax2 = ax.twinx()
ax2.bar(season_stats.index, season_stats['count'], alpha=0.4,
color='lime', width=0.5, label='Number of transfers')
ax2.set_ylabel('Количество трансферов')
ax2.legend(loc='upper right')
plt.tight_layout()
plt.show()
```

Листинг Б.8 — Интеграция рыночной стоимости

```
market_values['date_unix'] = pd.to_datetime(market_values['date_unix'],
errors='coerce')

market_values = market_values[market_values['value'] > 0]
df = df.sort_values('transfer_date')
market_values = market_values.sort_values('date_unix')

df = pd.merge_asof(
    df, market_values[['player_id', 'date_unix', 'value']],
    left_on='transfer_date', right_on='date_unix', by='player_id',
    direction='backward', tolerance=pd.Timedelta('365D')
)
df = df.rename(columns={'value': 'market_value'})
df['has_market_value'] = df['market_value'].notna()
df['temp_age_group'] = pd.cut(df['age'], bins=[0, 22, 27, 32, 100],
labels=['young', 'prime', 'peak', 'veteran'])

# Заполнение пропусков медианой по группе (позиция + возрастная группа)
market_value_medians = df[df['has_market_value']].groupby(['position',
'temp_age_group'])['market_value'].median().to_dict()
for idx, row in df[~df['has_market_value']].iterrows():
    key = (row['position'], row['temp_age_group'])
    if key in market_value_medians and not
pd.isna(market_value_medians[key]):
        df.loc[idx, 'market_value'] = market_value_medians[key]
    else:
        df.loc[idx, 'market_value'] =
df[df['has_market_value']]['market_value'].median()

df['market_value_log'] = np.log1p(df['market_value'])
df = df.drop(columns=['temp_age_group'])
```

Листинг Б.9 — Игровая статистика за 2 года до трансфера (часть 1)

```
player_performances['season_year']=player_performances['season_name'].apply(extract_season_year)

ALL_STATS=['goals','assists','minutes_played','yellow_cards','direct_red_cards','second_yellow_cards','own_goals','penalty_goals','goals_conceded','clean_sheets']

agg_dict = {stat:'sum' for stat in ALL_STATS if stat in player_performances.columns}

player_performances = calculate_advanced_metrics(player_performances)
advanced_metrics=['goals_assists_per_90','non_penalty_goals','clean_sheet_ratio']

for metric in advanced_metrics:
    if metric in player_performances.columns:
        agg_dict[metric] = 'mean'

perf_agg=player_performances.groupby(['player_id','season_year']).agg(agg_dict).reset_index()

stat_columns = list(agg_dict.keys())
```

Листинг Б.10 — Игровая статистика за 2 года до трансфера (часть 2)

```
stats = []
missing_stats = 0
for idx, row in df.iterrows():
    player_stats = perf_agg[(perf_agg['player_id'] == row['player_id']) & (perf_agg['season_year'] >= row['season_year'] - 2) & (perf_agg['season_year'] < row['season_year'])]
    row_stats = {}
    if len(player_stats) == 0:
        missing_stats += 1
        for stat in stat_columns:
            row_stats[stat] = 0
    else:
        for stat in stat_columns:
            if stat in ['goals_assists_per_90','clean_sheet_ratio']:
                row_stats[stat] = player_stats[stat].mean()
            else:
                row_stats[stat] = player_stats[stat].sum()
    stats.append(row_stats)
stats_df = pd.DataFrame(stats)
for col in stats_df.columns:
    df[f'{col}_last_2y'] = stats_df[col]
```

Листинг Б.11 — Травмы и выступления за сборную

```
player_injuries['from_date']
pd.to_datetime(player_injuries['from_date'], errors='coerce')
injury_stats = []
for idx, row in df.iterrows():
    inj = player_injuries[
        (player_injuries['player_id'] == row['player_id']) &
        (player_injuries['from_date'] <= row['transfer_date'])
    ]
    injury_stats.append(inj['days_missed'].sum())
df['injury_days'] = injury_stats

# Национальная сборная
agg_dict_national = {'goals': 'sum'}
for col in player_national.columns:
    if col not in ['player_id', 'season', 'season_name', 'date'] and col
not in agg_dict_national:
        if player_national[col].dtype in ['int64', 'float64']:
            agg_dict_national[col] = 'sum'

national_agg
player_national.groupby('player_id').agg(agg_dict_national).reset_index()
rename_dict = {col: f'national_{col}' if col != 'goals' else
'national_goals' for col in agg_dict_national.keys()}
national_agg = national_agg.rename(columns=rename_dict)
df = df.merge(national_agg, on='player_id', how='left')
for col in rename_dict.values():
    if col in df.columns:
        df[col] = df[col].fillna(0)
```


Листинг Б.12 — Финальный набор признаков и подготовка данных

```
performance_cols = [col for col in df.columns if col.endswith('_last_2y')]
df['position_simple'] = df['position'].fillna('Unknown').apply(lambda x:
x.split('-')[0] if pd.isna(x) and '-' in str(x) else str(x))

FEATURES = [
    'age', 'age_squared', 'market_value', 'market_value_log',
    'injury_days', 'national_goals', 'national_matches'
] + performance_cols

# Фильтровать только существующие столбцы
available_features = [f for f in FEATURES if f in df.columns]
# Удалите national_matches, если он не нужен
if 'national_matches' not in df.columns:
    available_features = [f for f in available_features if f !=
'national_matches']

# Убедитесь, что в df указана дата передачи
df['transfer_date'] = pd.to_datetime(df['transfer_date']) # уже должно
быть datetime, но на всякий случай

df_model = df[available_features + ['log_fee', 'season_year',
'transfer_fee', 'position_simple', 'transfer_date', 'player_id']].copy()

# Импутация пропусков
for col in available_features:
    if col in df_model.columns:
        if df_model[col].dtype in ['int64', 'float64']:
            median_val = df_model[col].median()
            df_model[col] = df_model[col].fillna(median_val)
        else:
            mode_val = df_model[col].mode()[0] if not
df_model[col].mode().empty else 'Unknown'
            df_model[col] = df_model[col].fillna(mode_val)

print(f"Финальный размер датасет: {len(df_model)} записей,
{len(available_features)} признаков")
```

Листинг Б.13 — Разделение данных по времени и масштабирование

```
train = df_model[df_model['transfer_date'].dt.year <= 2021]
test = df_model[df_model['transfer_date'].dt.year > 2021]

X_train = train[available_features]
y_train = train['log_fee']
X_test = test[available_features]
y_test = test['log_fee']

# Для реальных сумм (обратное преобразование)
y_train_real = np.expml(y_train)
y_test_real = np.expml(y_test)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Train size: {len(X_train)}, Test size: {len(X_test)}")
```

Листинг Б.14 — Подбор гиперпараметров

```
tscv = TimeSeriesSplit(n_splits=5)

# Ridge
ridge_params = {'alpha': [1.0, 10.0, 50.0]}
ridge_grid = GridSearchCV(Ridge(random_state=42, max_iter=1000),
ridge_params,
                        cv=tscv, scoring='r2', n_jobs=-1)
ridge_grid.fit(X_train_scaled, y_train)
best_ridge = ridge_grid.best_estimator_
print(f"Best Ridge alpha: {ridge_grid.best_params_}")

# Lasso
lasso_params = {'alpha': [0.001, 0.01, 0.1]}
lasso_grid = GridSearchCV(Lasso(random_state=42, max_iter=1000),
lasso_params, cv=tscv, scoring='r2', n_jobs=-1)
lasso_grid.fit(X_train_scaled, y_train)
best_lasso = lasso_grid.best_estimator_
print(f"Best Lasso alpha: {lasso_grid.best_params_}")

# Random Forest
rf_params = {
    'n_estimators': [100, 200],
    'max_depth': [15, 20],
    'min_samples_split': [5, 10]
}
rf_grid = GridSearchCV(RandomForestRegressor(random_state=42, n_jobs=-1),
rf_params, cv=tscv, scoring='r2', n_jobs=-1)
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_
print(f"Best RF params: {rf_grid.best_params_}")

# Gradient Boosting
gb_params = {
    'n_estimators': [100, 200],
    'learning_rate': [0.01, 0.05],
    'max_depth': [3, 5]
}
gb_grid = GridSearchCV(GradientBoostingRegressor(random_state=42,
subsample=0.8), gb_params, cv=tscv, scoring='r2', n_jobs=-1)
gb_grid.fit(X_train, y_train)
best_gb = gb_grid.best_estimator_
print(f"Best GB params: {gb_grid.best_params_}")
```

Листинг Б.15 — Обучение и сравнение моделей

```
models = {'Ridge (L2)': best_ridge, 'Lasso (L1)': best_lasso, 'Random
Forest': best_rf, 'Gradient Boosting': best_gb }

results_train = []
results_test = []
best_model = None
best_r2_test = -np.inf
for name, model in models.items():
    print(f"\nTraining: {name}")
    if name in ['Ridge (L2)', 'Lasso (L1)']:
        model.fit(X_train_scaled, y_train)
        y_pred_train = model.predict(X_train_scaled)
        y_pred_test = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred_train = model.predict(X_train)
        y_pred_test = model.predict(X_test)
    y_pred_train_real = np.expml(y_pred_train)
    y_pred_test_real = np.expml(y_pred_test)
    mae_train = mean_absolute_error(y_train_real, y_pred_train_real)
    r2_train = r2_score(y_train_real, y_pred_train_real)
    mape_train = mean_absolute_percentage_error(y_train_real,
y_pred_train_real)
    mae_test = mean_absolute_error(y_test_real, y_pred_test_real)
    r2_test = r2_score(y_test_real, y_pred_test_real)
    mape_test = mean_absolute_percentage_error(y_test_real,
y_pred_test_real)
    results_train.append({'Model': name, 'MAE_train (€)': mae_train,
'R2_train': r2_train, 'MAPE_train (%)': mape_train})
    results_test.append({'Model': name, 'MAE_test (€)': mae_test,
'R2_test': r2_test, 'MAPE_test (%)': mape_test})
    print(f"    Train: MAE = €{mae_train:,.0f}, R2 = {r2_train:.4f}, MAPE =
{mape_train:.1f}%")
    print(f"    Test : MAE = €{mae_test:,.0f}, R2 = {r2_test:.4f}, MAPE =
{mape_test:.1f}%")
    if r2_test > best_r2_test:
        best_r2_test = r2_test
        best_model = model
        best_model_name = name
```

Листинг Б.16 — Анализ важности признаков

```
# Анализ важности признаков

# Определяем, с какими данными работаем
use_scaled = 'Ridge' in best_model_name or 'Lasso' in best_model_name
X_test_best = X_test_scaled if use_scaled else X_test

# Встроенная важность для древовидных моделей
if hasattr(best_model, 'feature_importances_'):
    importance = pd.DataFrame({
        'Признак': available_features,
        'Важность': best_model.feature_importances_
    }).sort_values('Важность', ascending=False)

    print("\nТоп-10 важнейших признаков (feature_importances_):")
    for i, row in importance.head(10).iterrows():
        print(f"    {i+1:2d}. {row['Признак']:35s}: {row['Важность']:.4f}
({row['Важность']*100:.1f}%)")

# Permutation importance (для всех моделей)
print("\nPermutation importance анализ...")
perm = permutation_importance(
    best_model, X_test_best, y_test,
    n_repeats=10, random_state=42, scoring='r2', n_jobs=-1
)

perm_importance = pd.DataFrame({
    'Признак': available_features,
    'Важность': perm.importances_mean,
    'Std': perm.importances_std
}).sort_values('Важность', ascending=False)

print("Топ-10 по permutation importance:")
for i, row in perm_importance.head(10).iterrows():
    print(f"    {i+1:2d}. {row['Признак']:35s}: {row['Важность']:.4f} ±
{row['Std']:.4f}")
```

Листинг Б.17 — Анализ конкретных трансферов (часть 1)

```
def find_player(player_name):
    if player_profiles.empty:
        print("    Данные профилей не загружены")
        return None

    matches = player_profiles[
        player_profiles['player_name'].str.contains(player_name,
case=False, na=False)
    ]
    if len(matches) == 0:
        print(f"    Игрок '{player_name}' не найден")
        return None
    print(f"    Найдено {len(matches)} совпадений:")
    for i, (_, row) in enumerate(matches.iterrows()):
        print(f"        {i+1}. {row['player_name']} (ID: {row['player_id']})")
    return matches.iloc[0]

def analyze_player_transfer(player_id, target_year):
    transfer = df[
        (df['player_id'] == player_id) &
        (df['season_year'] == target_year)
    ]

    if len(transfer) == 0:
        print(f"    Трансфер {target_year} года не найден")

    # Показываем доступные трансферы
    player_transfers = df[df['player_id'] ==
player_id].sort_values('season_year')
    if len(player_transfers) > 0:
        print("    Доступные трансферы:")
        for _, t in player_transfers.iterrows():
            from_team = t.get('from_team_name', 'Unknown')
            to_team = t.get('to_team_name', 'Unknown')
            print(f"        • {t['season_year']}: {from_team} → {to_team}
(€{t['transfer_fee']:, .0f})")
        return None
```

Листинг Б.18 — Анализ конкретных трансферов (часть 2)

```
data = transfer.iloc[0]
profile_data = {}
for feat in available_features:
    profile_data[feat] = data.get(feat, 0)
profile = pd.DataFrame([profile_data])[available_features]
if use_scaled:
    profile_scaled = scaler.transform(profile)
    pred_log = best_model.predict(profile_scaled)[0]
else:
    pred_log = best_model.predict(profile)[0]
pred_value = np.expml(pred_log)
actual = data['transfer_fee']
if actual > 0:
    error_abs = abs(actual - pred_value)
    error_pct = ((error_abs / actual) * 100)
else:
    error_abs = 0
    error_pct = 0
player_info = player_profiles[player_profiles['player_id'] ==
player_id]
player_name = player_info.iloc[0]['player_name'] if len(player_info) >
0 else "Неизвестно"
return {
    'player_name': player_name,
    'season': data['season_year'],
    'age': data['age'],
    'position': data.get('position', 'Unknown'),
    'actual_fee': actual,
    'predicted_fee': pred_value,
    'error_abs': error_abs,
    'error': error_pct
}
```

Листинг Б.19 — Анализ конкретных трансферов (часть 3)

```
players_to_analyze = [
    ("Karim Benzema", 2009),
    ("Zlatan Ibrahimović", 2011),
    ("Antoine Griezmann", 2019),
    ("Luis Suárez", 2022),
    ("Kristjan Asllani", 2023),
    ("Arnau Tenas", 2025)
]

case_studies = []

print("Анализ известных трансферов")

for player_name, year in players_to_analyze:
    print(f"\nПоиск: {player_name} ({year})")
    player = find_player(player_name)

    if player is not None:
        result = analyze_player_transfer(player['player_id'], year)
        if result is not None:
            case_studies.append(result)

# Сводная таблица кейсов
case_df = pd.DataFrame(case_studies)
print("\nСводная таблица результатов анализа")

# Форматирование для вывода
display_df = case_df.copy()
display_df['actual_fee'] = display_df['actual_fee'].apply(lambda x:
f"€{x/1_000_000:.1f}M")
display_df['predicted_fee'] = display_df['predicted_fee'].apply(lambda x:
f"€{x/1_000_000:.1f}M")
display_df['error'] = display_df['error'].apply(lambda x: f"{x:.1f}%")

print(display_df[['player_name', 'season', 'actual_fee', 'predicted_fee',
'error']].to_string(index=False))
```